Project – Multivariate and Hierarchical Data

Part: Multivariate Methods

Olivier Thas





2024-2025

Contents

1 Introduction **5**

2 The Singular Value Decomposition **7**

2.1 SVD as a Matrix Approximation Method 7

2.2 Geometric Interpretation of SVD 24

2.3 The SVD of the inverse of a symmetric square matrix 28

2.4 Multidimensional Scaling 29

2.4.1 Introduction 29

2.4.2 MDS via the SVD 31

2.4.3 Examples 35

2.5 The Biplot 40

2.6 Principal Component Analysis 46

2.6.1 Introduction 46

2.6.2 Traditional Formulation of PCA 47

2.6.3 Relation between PCA and SVD 53

2.6.4 Conservation of Variance 56

2.6.5 Choosing the number of dimensions 57

2.6.6 Covariance versus correlation matrix 60

2.6.7 PCA and the multivariate normal distribution 64

2.6.8 Biplots for PCA 68

2.7 Summary of the SVD 70

2.8 A Sparse Singular Value Decomposition 72

2.8.1 Introduction 72

2.8.2 Penalised Matrix Decomposition 77

2.8.3 Sparse PCA via PMD 79

3 Linear Discriminant Analysis **86**

4 Cluster Analysis **101**

4.1 Introduction 101
4.2 Graphical Diagnostic Tools 103
4.2.1 Clusplot 104
4.2.2 Silhouette Plot 105
4.3 Partition-based clustering 107
4.3.1 K-means methods 108
4.3.2 Pam 111
4.4 Hierarchical Cluster Analysis 116
4.4.1 Agnes 116
4.4.2 Cluster Tree 119
4.4.3 Ward's Method 127
4.5 Determination of the Number of Clusters 128
4.5.1 Introduction 128
4.5.2 Scree Plot 128

1 Introduction

Introduction to Multivariate and High-Dimensional Data

- Multivariate statistical methods have a long history, but were designed for the $n >>> p$ case
- Nowadays many high throughput technologies generate multivariate data with many variables (large p) as compared to the number of independent replicates or samples (sample size n), resulting in high-dimensional data, which is characterised by

$$p >>> n.$$

- New statistical methods for dealing with the $p >>> n$ case have been developed in the last 20 years. Some of them are adaptations of multivariate methods.

There is not always a clear distinction between high dimensional data and multivariate data. The latter is an older term and many methods were developed for multivariate data. Only later, with modern data generating technologies the dimensionality of the datasets started to grow rapidly. Because of the need to have meaningful and efficient data analysis methods for these new kind of datasets, methods started to being developed specifically for high dimensional data. There is of course a grey zone in between. Examples of high dimensional data technologies include next generation sequencing (e.g. RNASeq, microbiome data analysis), microarrays (e.g. gene expression, SNP), satellite images, ...

Issues with high-dimensional data:

- Computational problems: large matrices, numerical accuracy of computers become an issue
- Classical asymptotic theory does not hold for $p \to \infty$ as $n \to \infty$
- Model (or feature) selection requires specialised methods to deal with the enormous number of possible models. (e.g. in linear regression with p potential predictors: 2^p possible models)
- Models that can potentially depend on large-p predictors are vulnerable to huge overfitting.
- In searching for associations between an outcome and large p potential exploratory variables, we are at risk to make many false discoveries
- The Curse of Dimensionality may cause a prediction model to become useless. (n data points become sparse in large-p dimensional space)

Content of the course

- Exploring large data sets:
  - Singular value decomposition (SVD), MDS, PCA, biplots, LDA
  - Extensions of SVD, MDS, PCA and LDA to high-dimensional settings
  - Cluster analysis (unsupervised learning)

---

2 The Singular Value Decomposition

2.1 SVD as a Matrix Approximation Method

The Singular Value Decomposition

- The SVD is a very important method in Data Sciences and in Engineering.
- We first introduce SVD as a matrix approximation method
- Next we show how this leads to PCA, MDS and biplots as methods for data exploration and dimensional reduction

---

Some notation:

Let $\boldsymbol{X}$ denote an $n \times p$ data matrix:

- n data entries on
- p variables (or features)

Let x_{ij} denote the ith observation on variable j.

We often use $\boldsymbol{x}_i^t = (x_{i1}, x_{i2}, \ldots, x_{ip})$ to denote the ith observation vector.

In some applications the matrix $\boldsymbol{X}$ does not have the structure of a data matrix (observations in rows/variables in columns), but it is just a matrix with data entries (symmetrical role of rows and columns).

We adopt the convention that vectors are always column vectors. For example, $\boldsymbol{x}_i^t$ is the ith row, but because of the convention $\boldsymbol{x}_i$ is a column vector (note the use of the transpose operator).

Singular Value Decomposition

Let $\boldsymbol{X}$ be an $n \times p$ matrix of rank $r \leq \min(n, p)$. Then there exists a SVD

$$\boldsymbol{X} = \sum_{j=1}^{r} \delta_j \boldsymbol{u}_j \boldsymbol{v}_j^t,$$

with

- singular values $\delta_1 \geq \delta_2 \geq \cdots \geq \delta_r > 0$
- orthonormal left-singular vectors $\boldsymbol{u}_j$ of length n $(\boldsymbol{u}_j^t \boldsymbol{u}_k = \delta_{jk})$
- orthonormal right-singular vectors $\boldsymbol{v}_j$ of length p $(\boldsymbol{v}_j^t \boldsymbol{v}_k = \delta_{jk})$

In the previous slide, the notation δ_{jk} stands for the indicator $\mathrm{I}\{j = k\}$.

Note on the rank of a matrix

The rank of a matrix is the dimension of the vector space spanned by the columns of the matrix, or, equivalently, the dimension of the vector space spanned by the rows of the matrix. Here is a simple example:

$$\boldsymbol{X} = \begin{pmatrix} 3 & 6 \\ 1 & 2 \\ 0.23 & 0.46 \\ -2 & -4 \end{pmatrix}$$

The columns are linearly related: the second column equals twice the first. Geometrically, the rows of the matrix are represented by points in a two-dimensional vector space (columns of matrix); the points lie exactly on a line (= one-dimensional subspace). Hence the rank of the matrix is 1.

SVD is a decomposition

Note that $\boldsymbol{X} = \sum_{j=1}^{r} \delta_j \boldsymbol{u}_j \boldsymbol{v}_j^t$ should be read as a decomposition: it is a sum of matrices. Each term has a scalar factor (δ_i) and a $n \times p$ matrix formed by the multiplication of the n-vector $\boldsymbol{u}_j$ and the p-vector $\boldsymbol{v}_j^t$.

Since the singular vectors are normalised we could roughly say that all $\boldsymbol{u}_j \boldsymbol{v}_j^t$ have entries of about the same magnitude. Since the singular values δ_j are ordered from large to small, we could argue that the first term in the decomposition is the most contributing, the second the second most contributing, etc.

Note that the terms $\delta_j \boldsymbol{u}_j \boldsymbol{v}_j^t$ are $n \times p$ matrices (hence "composition"). In particular, δ_j are scalar weights, and $\boldsymbol{u}_j \boldsymbol{v}_j^t$ are $n \times p$ matrices.

In matrix notation the SVD is written as

$$\boldsymbol{X} = \boldsymbol{U}\boldsymbol{D}\boldsymbol{V}^t,$$

where

- $\boldsymbol{U}$ is an $n \times r$ matrix with columns $\boldsymbol{u}_1, \ldots, \boldsymbol{u}_r$
- $\boldsymbol{V}$ is a $p \times r$ matrix with columns $\boldsymbol{v}_1, \ldots, \boldsymbol{v}_r$
- $\boldsymbol{D}$ is a diagonal matrix with elements $\delta_1, \ldots, \delta_r$.

If $\boldsymbol{X}$ is a symmetric matrix, then $\boldsymbol{U} = \boldsymbol{V}$.

The importance of the SVD is tied to the following property.

SVD: optimal approximation

Let $\boldsymbol{X}$ be an $n \times p$ matrix of rank $r \leq \min(n, p)$, and let $\boldsymbol{A}$ denote an $n \times p$ matrix of rank $k \leq r$, with elements denoted by a_{ij}.

The matrix $\boldsymbol{A}$ of rank $k \leq r$ that minimises

$$\sum_{i=1}^{n} \sum_{j=1}^{p} (x_{ij} - a_{ij})^2$$

is given by the truncated SVD

$$\boldsymbol{X}_k = \sum_{j=1}^{k} \delta_j \boldsymbol{u}_j \boldsymbol{v}_j^t.$$

- The SVD finds the matrix $\boldsymbol{A}$ that minimises

  $$\sum_{i=1}^{n} \sum_{j=1}^{p} (x_{ij} - a_{ij})^2.$$

  This sum can be looked at as a total sum of squared errors between the elements of $\boldsymbol{X}$ and $\boldsymbol{A}$.

- The truncated SVD has $k < r$ terms. Hence, generally $\boldsymbol{X}_k$ does not coincide with $\boldsymbol{X}$. It is considered as an approximation.

Hence

$$\boldsymbol{X}_k = \sum_{j=1}^{k} \delta_j \boldsymbol{u}_j \boldsymbol{v}_j^t$$

is the best rank k approximation of $\boldsymbol{X}$ in terms of the Frobenius norm

$$\sum_{i=1}^{n}\sum_{j=1}^{p}(x_{ij} - x_{kij})^2 = \|\boldsymbol{X} - \boldsymbol{X}_k\|_F^2$$

(kind of a *total least squares error*).

Mathematically we write

$$\boldsymbol{X}_k = \text{ArgMin}_{A:\text{rank}(A)=k}\|\boldsymbol{X} - \boldsymbol{A}\|_F^2.$$

Hence, the truncated SVD $\boldsymbol{X}_k$ is the best rank-k approximation of $\boldsymbol{X}$ (in terms of the Frobenius norm). Note that the norm is symmetric w.r.t. rows and columns.

The truncated SVD may also be written as

$$\boldsymbol{X}_k = \boldsymbol{U}_k \boldsymbol{D}_k \boldsymbol{V}_k^t$$

with

- $\boldsymbol{U}_k = (\boldsymbol{u}_1, \boldsymbol{u}_2, \ldots, \boldsymbol{u}_k)$, an $n \times k$ matrix
- $\boldsymbol{V}_k = (\boldsymbol{v}_1, \boldsymbol{v}_2, \ldots, \boldsymbol{v}_k)$, a $p \times k$ matrix
- $\boldsymbol{D}_k$ a $k \times k$ diagonal matrix with elements $\delta_1, \delta_2, \ldots, \delta_k$.

Some informal statements about the truncated SVD

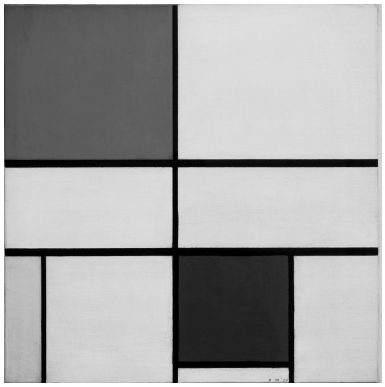
$$\boldsymbol{X}_k = \sum_{j=1}^{k} \delta_j \boldsymbol{u}_j \boldsymbol{v}_j^t.$$

- It can be considered as a weighted sum of matrices $\boldsymbol{u}_j \boldsymbol{v}_j^t$, with weights δ_j.
- The terms are ordered with decreasing weights $\delta_1 \geq \delta_2 \geq \cdots \geq \delta_k > 0$.
- The matrices $\boldsymbol{u}_j \boldsymbol{v}_j^t$ are of equal "magnitude" (constructed from normalised vectors).
- Truncation at k results in k δ_j's, $k \times n$ elements in the $\boldsymbol{u}_j$ and $k \times p$ elements in the $\boldsymbol{v}_j$. Hence a total of $k + kn + kp = k(1 + n + p)$ elements (usually much smaller than np). (Note that restrictions apply to $\boldsymbol{u}_j$ and $\boldsymbol{v}_j$; hence even less independent elements).

  $\longrightarrow$ data compression

Example

Have a look at this painting of Mondriaan (1872 – 1944), here shown in black-and-white.



This picture can be represented as a 2054×2043 matrix $\boldsymbol{X}$ with gray scale intensities $\in [0, 1]$. ($\approx 4 \times 10^6$ data entries)

Example

```
> library(jpeg)
> img<-readJPEG("../Data/Mondriaan1.jpg")
>
> plot(1:2,type="n")
> rasterImage(img,1, 1, 2, 2)
>
> # extract one channel
>       (three colour channels in img object)
> X<-img[,,1]
> dim(X)
[1] 2054 2043
```

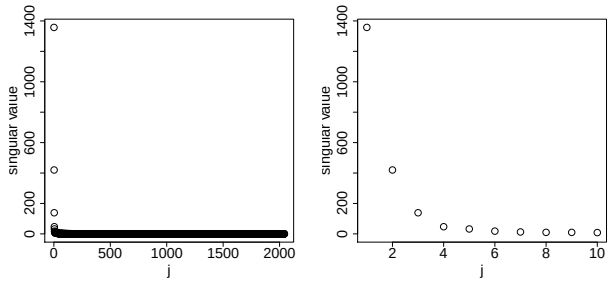
The object img is a three-dimensional array. It contains three colour channels, but here we only look at one channel (as in black-and-white). The single channel is extracted as img[,,1].

The rank of the $\boldsymbol{X}$ matrix is $\min(2054, 2043) = 2043$.

Example

We look at the singular values.

```
> X.svd<-svd(X)
> plot(X.svd$d,xlab="j",ylab="singular value")
> plot(X.svd$d,xlim=c(1,10),xlab="j",ylab="singular value")
```

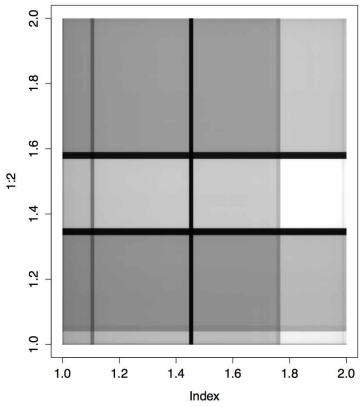


The singular values decrease very rapidly.

Example

We look at an approximation truncated after $k = 1$ term.

```
u<-X.svd$u
v<-X.svd$v
d<-diag(X.svd$d)
u1 <- as.matrix(u[, 1])
v1 <- as.matrix(v[, 1])
d1 <- as.matrix(d[1, 1])
X1 <- u1 %*% d1 %*% t(v1)
X1[X1>1]<-1
X1[X1<0]<-0
img1<-img
for(i in 1:3) {
  img1[,,i]<-X1
}
plot(1:2,type="n")
rasterImage(img1,1, 1, 2, 2)
```



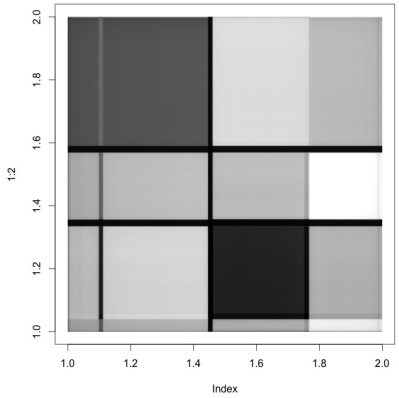
- The first order truncated SVD does not give an exact regeneration of the original matrix (painting), but the rough structure is visible.
- The original matrix contains $2054 \times 2043 \approx 4 \times 10^6$ data entries, whereas the truncated SDV gives a matrix approximation with $2054 + 2043 + 1 = 4098$ data entries (actually a few less because of the orthonormality constraints).

  This is a data compression to $1 - \frac{2054+2043+1}{2054\times 2043} \approx 0.1\%$!

Example

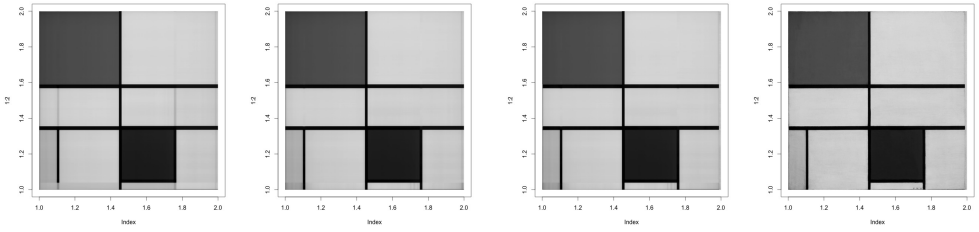
We look at an approximation truncated after $k = 2$ terms.

```
k<-2
Uk<- as.matrix(u[, 1:k])
Vk<- as.matrix(v[, 1:k])
Dk <- as.matrix(d[1:k, 1:k])
Xk <- Uk %*% Dk %*% t(Vk)
Xk[Xk>1]<-1
Xk[Xk<0]<-0
img1<-img
for(i in 1:3) {
  img1[,,i]<-Xk
}
plot(1:2,type="n")
rasterImage(img1,1, 1, 2, 2)
```



Example

Approximations truncated after $k = 3, 4, 5$ terms (final graph is original).

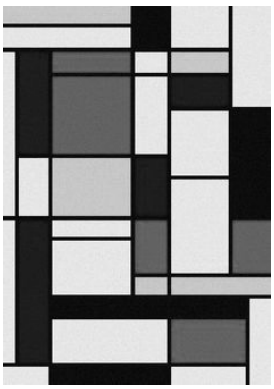


The SVD truncated after $k = 5$ terms looks like a good approximation. This gives a data reduction of $1 - \frac{5\times(2054+2043+1)}{2054\times2043} \approx 0.5\%$.

Thus with only $k = 5$ out of the rank$(\boldsymbol{X}) = 2043$ terms in the SVD a very good approximation is found; that is $5/2043 = 0.2\%$ of all dimensions of the vector space of the matrix. The reason is that the original shows a structure that obeys quite well the row and column structure of a matrix (vertical and horizontal lines) with between the lines large monochrome planes.

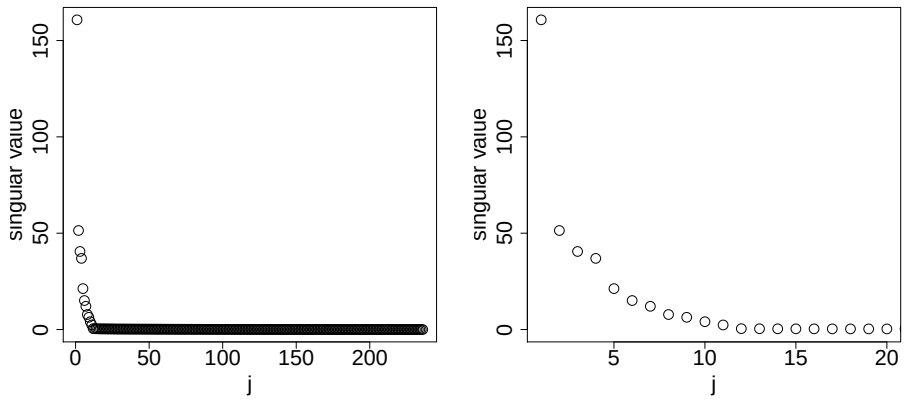
Example

We repeat the analysis, but now starting from another Mondriaan painting.



The jpeg file has resolution 334×236 (≈ 80000 data entries)

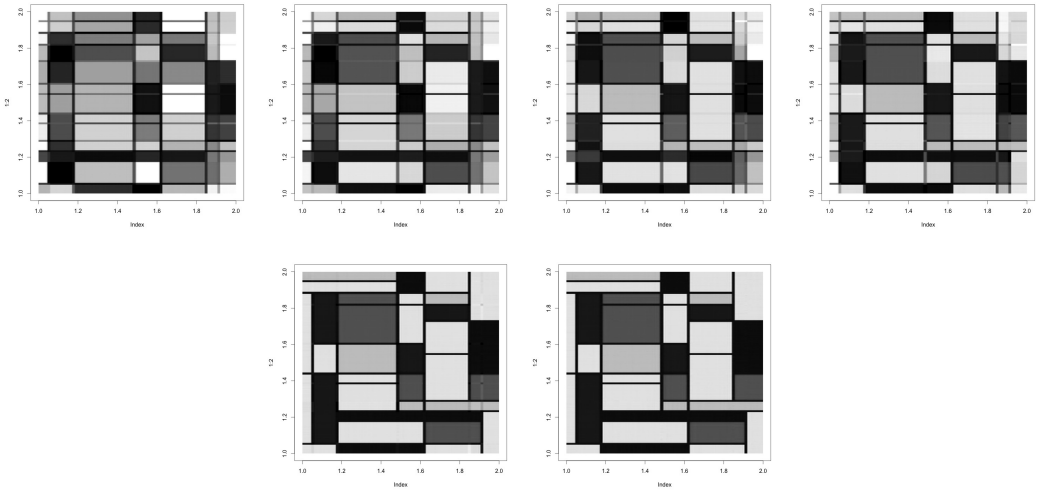
Example



The singular values still decay rapidly, but slower than for the previous painting. The reason is that the new painting shows a bit more complexity than the previous (more horizontal and vertical lines).

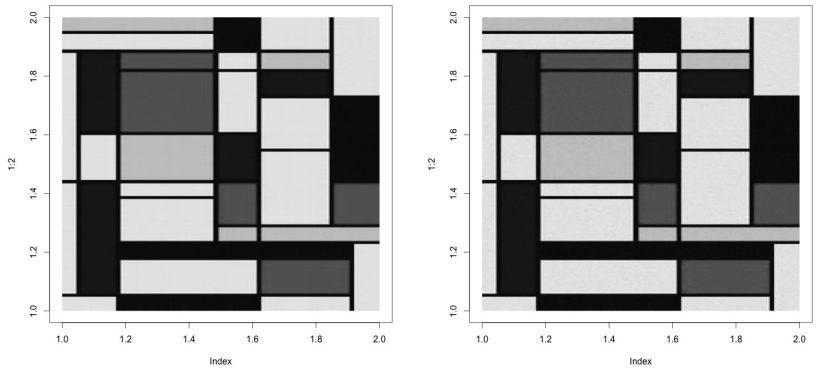
Example

Approximations truncated after $k = 3, 4, 5, 6, 10, 15$ terms.



Example

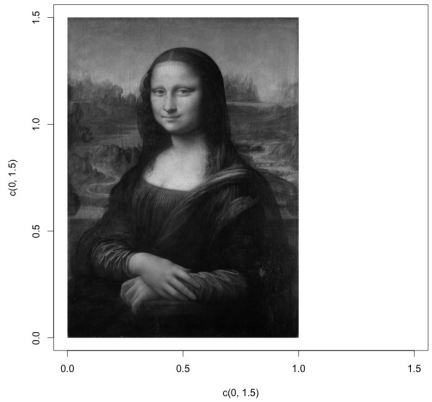
Finally with truncations after 20 terms (final graph is original).



With $k = 3$ the data compression is $1 - \frac{3\times(334+236+1)}{334\times236} \approx 2.5\%$. With $k = 20$ (good approximation) it is $1 - \frac{20\times(334+236+1)}{334\times236} \approx 17\%$; that is $20/236 \approx 8.5\%$ of all dimensions of the vector space of the matrix.

Example

Finally an example of a painting with some more complexity.

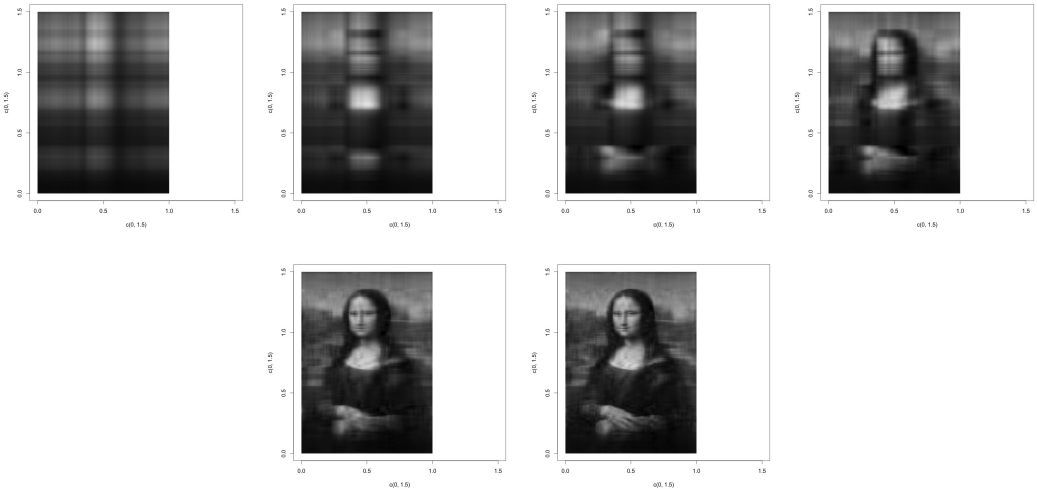


The jpeg file has resolution 1024×687 (≈ 700000 data entries).

The Mona Lisa was painted by Leonardo da Vinci between 1503 and 1506 (perhaps up to 1517).

Example

Approximations truncated after $k = 1, 2, 3, 5, 10, 15$ terms.



With $k = 1$ the data compression is $1 - \frac{1024+687+1}{1024\times687} \approx 0.2\%$. With $k = 15$ it is $1 - \frac{15\times(1024+687+1)}{1024\times687} \approx 3.6\%$.

Example

Approximations truncated after $k = 20, 30, 50, 100$ terms (final graph is the orginal).



With $k = 20$ the data compression is $1 - \frac{20\times(1024+687+1)}{1024\times687} \approx 5\%$. With $k = 100$ it is $1 - \frac{100\times(1024+687+1)}{1024\times687} \approx 25\%$; that is $100/687 \approx 15\%$ of all dimensions of the vector space of the matrix.

A more complicated matrix needs more terms in the SVD to give good approximations.

The previous examples demonstrate that the SVD works well for matrix approximation, and that the order k at which the truncation is applied depends on the complexity of the picture.

- Later we discuss methods to find a good choice of k.
- Next we show how the SVD can be used in data exploration

All examples were realistic, but the $\boldsymbol{X}$ matrix had not the typical structure of a data matrix that we are used to in statistics. Here the row and columns can be interchanged; the result will still be the painting, but rotated. In statistics a data matrix has typically observations/subjects/experimental units in the rows, and variables in the columns, resulting in an asymmetric matrix structure.

2.2 Geometric Interpretation of SVD

Geometric Interpretation of SVD

In data sciences the matrix $\boldsymbol{X}$ often represents a data matrix with

- n data entries on
- p variables (or features)

Let x_{ij} denote the ith observation on variable j.

Data are often represented in a p-dimensional space: each p-dimensional observation $\boldsymbol{x}_i$ is a vector in a p-dimensional vector space $\mathbb{R}^p$.

We often use $\boldsymbol{x}_i^t = (x_{i1}, x_{i2}, \ldots, x_{ip})$ to denote the ith observation vector.

Other terms often used for the rows of a data matrix:

- rows
- data points
- subjects (often referring to persons/patients).

Write the truncated SVD as

$$\boldsymbol{X}_k = \boldsymbol{U}_k \boldsymbol{D}_k \boldsymbol{V}_k^t = \boldsymbol{Z}_k \boldsymbol{V}_k^t$$

with

$$\boldsymbol{Z}_k = \boldsymbol{U}_k \boldsymbol{D}_k$$

an $n \times k$ matrix.

Each of the n rows of $\boldsymbol{Z}_k$, say $\boldsymbol{z}_{ki}^t$, represents a point in a k-dimensional space.

Because of the orthonormality of the singular vectors, we also have

$$\begin{aligned} \boldsymbol{X}_k \boldsymbol{V}_k &= \boldsymbol{Z}_k \boldsymbol{V}_k^t \boldsymbol{V}_k \\ \boldsymbol{X}_k \boldsymbol{V}_k &= \boldsymbol{Z}_k. \end{aligned}$$

Thus the matrix $\boldsymbol{V}_k$ is a transformation matrix that may be used to transform $\boldsymbol{X}_k$ into $\boldsymbol{Z}_k$, and $\boldsymbol{Z}_k$ into $\boldsymbol{X}_k$.

The matrix $\boldsymbol{V}_k$ transforms the p-dimensional $\boldsymbol{X}_k$ into the k-dimensional $\boldsymbol{Z}_k$: $\boldsymbol{Z}_k = \boldsymbol{X}_k \boldsymbol{V}_k$. Note, however, that the matrix $\boldsymbol{X}_k$ must not necessarily be used for this transformation, because the SVD of the original matrix $\boldsymbol{X}$ also gives directly $\boldsymbol{Z}_k = \boldsymbol{U}_k \boldsymbol{D}_k$.

The inverse transformation from the k-dimensional $\boldsymbol{Z}_k$ to the p-dimensional $\boldsymbol{X}_k$ is given by the transpose of $\boldsymbol{V}_k$: $\boldsymbol{Z}_k \boldsymbol{V}_k^t = \boldsymbol{X}_k$. Often inverse transformations are given by the inverse of a matrix, but thanks to the orthonormality of the columns of $\boldsymbol{V}_k$, we get $\boldsymbol{V}_k^t \boldsymbol{V}_k = \boldsymbol{I}$, and thus $\boldsymbol{V}_k^t$ acts as an inverse.

Note that the transformation from the k-dimensional $\boldsymbol{Z}_k$ to the p-dimensional $\boldsymbol{X}_k$ is transforming points from a low dimensional space (k) to a high dimensional space (p). You may not interpret this as if this transformation adds information; the transformed points in $\boldsymbol{X}_k$ still live in a k-dimensional subspace of the larger p-dimensional space; the matrix $\boldsymbol{X}_k$ is only of rank k and thus contains less information than the original data matrix $\boldsymbol{X}$ (if rank($\boldsymbol{X}$)$= r > k$).

More importantly, it can be shown that (thanks to orthonormality of $\boldsymbol{V}$)

$$\boldsymbol{X}\boldsymbol{V}_k = \boldsymbol{Z}_k.$$

This follows from (w.l.g. rank($\boldsymbol{X}$)=p)

$$\begin{aligned}
\boldsymbol{X}\boldsymbol{V}_k &= \boldsymbol{U}\boldsymbol{D}\boldsymbol{V}^t\boldsymbol{V}_k = \boldsymbol{U}\boldsymbol{D}\begin{pmatrix}\boldsymbol{v}_1^t\\ \vdots \\ \boldsymbol{v}_p^t\end{pmatrix}\left(\boldsymbol{v}_1 \dots \boldsymbol{v}_k\right) \\
&= \boldsymbol{U}\boldsymbol{D}\boldsymbol{V}^t\boldsymbol{V}_k = \boldsymbol{U}\boldsymbol{D}\begin{pmatrix}1 & 0 & \dots & 0\\ 0 & 1 & \dots & 0\\ \vdots & \vdots & \ddots & 0\\ 0 & 0 & \dots & 1\\ 0 & 0 & \dots & 0\\ \vdots & \vdots & \vdots & \vdots\\ 0 & 0 & \dots & 0\end{pmatrix} = \boldsymbol{U}_k\boldsymbol{D}_k = \boldsymbol{Z}_k
\end{aligned}$$

We take a closer look at

$$\boldsymbol{Z}_k = \boldsymbol{X}\boldsymbol{V}_k = \begin{pmatrix} \boldsymbol{x}_1^t \\ \vdots \\ \boldsymbol{x}_n^t \end{pmatrix} \left(\boldsymbol{v}_1 \dots \boldsymbol{v}_k\right).$$

The ith row (observation) in $\boldsymbol{Z}_k$ equals

$$\boldsymbol{z}_{ki}^t = \boldsymbol{x}_i^t\boldsymbol{V}_k = \left(\boldsymbol{x}_i^t\boldsymbol{v}_1, \boldsymbol{x}_i^t\boldsymbol{v}_2, \dots, \boldsymbol{x}_i^t\boldsymbol{v}_k\right).$$

Hence, $\boldsymbol{z}_{ki}^t = \boldsymbol{x}_i^t\boldsymbol{V}_k$ is the orthogonal projection of $\boldsymbol{x}_i$ onto the k-dimensional subspace spanned by the columns of $\boldsymbol{V}_k$.

SVD transforms data set to lower dimensional data set

The SVD thus gives a transformation of the p dimensional data to $k \leq r$ dimensional data:

$$\boldsymbol{Z}_k = \boldsymbol{X}\boldsymbol{V}_k.$$

This is essentially a dimension reduction.

The transformation from p-dimensional $\boldsymbol{X}$ to k-dimensional $\boldsymbol{Z}_k$ is important. It shows that the n points in the rows of $\boldsymbol{Z}_k$ are the result of projecting the n points in $\boldsymbol{X}$ onto the columns of $\boldsymbol{V}_k$ (i.e. the first k singular vectors of $\boldsymbol{X}$). We say that the space of $\boldsymbol{Z}_k$ is spanned by the column of $\boldsymbol{V}_k$.

The points (rows) in $\boldsymbol{X}$ live in a p-dimensional space (or rank($\boldsymbol{X}$)$= r$ if $r < p$) and they are thus projected onto a lower dimensional space. This is in contrast to the projection $\boldsymbol{X}_k\boldsymbol{V}_k = \boldsymbol{Z}_k$, because the points in $\boldsymbol{X}_k$ live in a k-dimensional subspace of $\boldsymbol{X}$.

Note that with $k < r$ there is no unique transformation to transform $\boldsymbol{Z}_k$ back to $\boldsymbol{X}$. On the previous slide we only established the transformation $\boldsymbol{Z}_k\boldsymbol{V}_k^t = \boldsymbol{X}_k$. Indeed, starting from $\boldsymbol{X}\boldsymbol{V}_k = \boldsymbol{Z}_k$, and right-multiplying with $\boldsymbol{V}_k^t$ does not give the backtransformation, because $\boldsymbol{V}_k\boldsymbol{V}_k^t$ is not the identity matrix.

2.3 The SVD of the inverse of a symmetric square matrix

The SVD of the inverse of a symmetric square matrix

Let $\boldsymbol{A}$ denote a symmetric $p \times p$ square matrix of rank p (full rank). It has SVD

$$\boldsymbol{A} = \boldsymbol{V}\boldsymbol{D}\boldsymbol{V}^t.$$

its inverse exists and it can be shown that the inverse has SVD

$$\boldsymbol{A}^{-1} = \boldsymbol{V}\boldsymbol{D}^{-1}\boldsymbol{V}^t.$$

This is very easy to check. It should hold that $\boldsymbol{A}\boldsymbol{A}^{-1} = \boldsymbol{I}$. Let's try:

$$\boldsymbol{A}\boldsymbol{A}^{-1} = \boldsymbol{V}\boldsymbol{D}\boldsymbol{V}^t\boldsymbol{V}\boldsymbol{D}^{-1}\boldsymbol{V}^t = \boldsymbol{V}\boldsymbol{D}\boldsymbol{D}^{-1}\boldsymbol{V}^t = \boldsymbol{V}\boldsymbol{V}^t = \boldsymbol{I}.$$

2.4 Multidimensional Scaling

2.4.1 Introduction

Multidimensional Scaling

The objective of Multidimensional Scaling (MDS) is to find a low-dimensional representation, say k-dimensional, of n data points such that the distances between the n points in the k-dimensional space is a good approximation of a given squared distance matrix, say $\boldsymbol{D}_X$.

- The squared distance matrix $\boldsymbol{D}_X$ may be given without knowledge of the original observations (not even the dimensionality), or it may be computed from a given set of n p-dimensional data points.
- Note that the distances between points in a k-dimensional subspace coincide with the distances between these points in the larger p-dimensional space.

MDS may be useful in the following two settings:

- A high dimensional data matrix $\boldsymbol{X}$ is given, and one wants to get a visual representation (in 2 or 3 dimensions) of the observations. In this graph each point represents an observation (row of data matrix). From this graph one wants points close to one another to be similar, and observations far away from one another to be dissimilar. Thus the distances between the n points in the original p-dimensional space should be well preserved in the 2 or 3 dimensional space.
- In some applications the researcher only has knowledge of the similarity (or dissimilarity) between observations. For example, food products can be evaluated by a taste panel and a dissimilarity matrix can be completed. This dissimilarity matrix shows for each pair of food products their dissimilarity (numerical values provided by taste panel, but not objectively quantified). Given this dissimilarity matrix, a 2 or 3 dimensional graph could be helpful if the distances between the points (food products) in this graph are monotonically related to the dissimilarities provided by the taste panel. Food products close to one another in this graph, taste similarly.

Here we discuss the "metric" MDS, which actually requires the Euclidean distances between observations. However, the method works also well if the matrix $\boldsymbol{D}_X$ contains dissimilarities rather than squared Euclidean distances.

In this chapter we assume that the data matrix $\boldsymbol{X}$ is column-centered (i.e. each column has mean zero).

Centering can be accomplished by multiplying the original data matrix $\boldsymbol{X}$ with the $n \times n$ centering matrix

$$\boldsymbol{H} = \boldsymbol{I} - \frac{1}{n}\mathbf{1}\mathbf{1}^t,$$

in which $\mathbf{1}$ is an n-vector with all entries equal to 1. Hence, $\boldsymbol{H}\boldsymbol{X}$ is the column-centered data matrix.

We will assume that $\boldsymbol{X}$ is already column-centered, and therefore $\boldsymbol{H}\boldsymbol{X} = \boldsymbol{X}$.

Note that the matrix $\mathbf{1}\mathbf{1}^t$ is an $n \times n$ matrix with all entries set to 1.

2.4.2 MDS via the SVD

We will need the following interesting relationship between the Gram matrix $\boldsymbol{X}\boldsymbol{X}^t$ and the distance matrix.

- For an $n \times p$ data matrix $\boldsymbol{X}$, the matrix $\boldsymbol{D}_X$ with squared distances has elements

$$(\boldsymbol{x}_i - \boldsymbol{x}_j)^t(\boldsymbol{x}_i - \boldsymbol{x}_j) = \|\boldsymbol{x}_i\|^2 - 2\boldsymbol{x}_i^t\boldsymbol{x}_j + \|\boldsymbol{x}_j\|.^2$$

- The $n \times n$ squared distance matrix can then be written as

$$\boldsymbol{D}_X = \boldsymbol{N} - 2\boldsymbol{X}\boldsymbol{X}^t + \boldsymbol{N}^t,$$

  with $\boldsymbol{N}$ the $n \times n$ matrix with ith row filled with $\|\boldsymbol{x}_i\|^2$.

- Note that the elements of $\boldsymbol{N}$ can also be found on the diagonal of $\boldsymbol{X}\boldsymbol{X}^t$.

- Given the structure of the $\boldsymbol{H}$ and $\boldsymbol{N}$ matrices, it is easy to verify that

$$-\frac{1}{2}\boldsymbol{H}\boldsymbol{D}_X\boldsymbol{H} = \boldsymbol{X}\boldsymbol{X}^t.$$

  This gives an important relation between the distance matrix and the Gram matrix. We will use the notation $\boldsymbol{G}_X = -\frac{1}{2}\boldsymbol{H}\boldsymbol{D}_X\boldsymbol{H}$.

The $n \times n$ Gram matrix $\boldsymbol{X}\boldsymbol{X}^t$ equals

$$\begin{pmatrix} \boldsymbol{x}_1^t \\ \vdots \\ \boldsymbol{x}_n^t \end{pmatrix} \begin{pmatrix} \boldsymbol{x}_1 \dots \boldsymbol{x}_n \end{pmatrix}$$

and has thus on its (i, j)th position the inner product

$$\boldsymbol{x}_i^t\boldsymbol{x}_j = \|\boldsymbol{x}_i\|\|\boldsymbol{x}_j\| \cos < \boldsymbol{x}_i, \boldsymbol{x}_j > .$$

On the diagonal we find $\boldsymbol{x}_i^t\boldsymbol{x}_i = \|\boldsymbol{x}_i\|^2$, the squared norm of the ith observation.

The relation $-\frac{1}{2}\boldsymbol{H}\boldsymbol{D}_X\boldsymbol{H} = \boldsymbol{X}\boldsymbol{X}^t$ tells us that the distance matrix and the Gram matrix contain the same information. The distances, however, are in most situations easier to interpret.

The truncated SVD of $\boldsymbol{X}$ minimises the Frobenius norm $\|\boldsymbol{X} - \boldsymbol{X}_k\|_F^2$, and this has an important consequence.

Consider two distance matrices:

- Let $\boldsymbol{D}_X$ denote the $n \times n$ matrix with the squared Euclidian distances between the n data points $\boldsymbol{x}_i$ in the original p-dimensional space.
- Let $\boldsymbol{D}_{Zk}$ denote the $n \times n$ matrix with the squared Euclidian distances between the n transformed data points $\boldsymbol{z}_{ki}$ in the reduced k-dimensional space.

Then, it can be shown that the truncated SVD also minimises

$$\|\boldsymbol{D}_X - \boldsymbol{D}_{Zk}\|_F^2.$$

Hence, the n points $\boldsymbol{z}_{ki}$ in the k-dimensional subspace spanned by the columns of $\boldsymbol{V}_k$ are the best approximation of the n original points $\boldsymbol{x}_i$ in the original p-dimensional space in the sense that the Euclidean distances between the n original points are best approximated by the Euclidean distances between the n transformed points, among all possible k-dimensional linear subspaces.

Before continuing we need a more formal notation. Let

$$\boldsymbol{D}(\boldsymbol{X})$$

denote the $n \times n$ matrix of squared Euclidean distances between the n rows of the matrix $\boldsymbol{X}$.

It can be shown that the truncated SVD also gives

$$\boldsymbol{X}_k \;=\; \text{ArgMin}_{A:\text{rank}(A)=k} \|D(\boldsymbol{X}) - \boldsymbol{D}(\boldsymbol{A})\|_F^2.$$

Note, however, that the calculation of $\boldsymbol{X}_k$ as the truncated SVD of $\boldsymbol{X}$ requires the knowledge of $\boldsymbol{X}$.

We can go one step further, and find a k-dimensional representation of the n data points that approximate a given distance matrix, say $\boldsymbol{D}_X$, without knowing the data matrix $\boldsymbol{X}$ of the p-dimensional data points.

It is important to understand the meaning of the relation between the truncated SVD and the approximation of $\boldsymbol{D}(\boldsymbol{X})$.

First note that the distance matrices calculated from $\boldsymbol{Z}_k$ and from $\boldsymbol{X}_k$ coincide. Since we know that distance matrices are determined by Gram matrices, it is sufficient to show that the Gram matrices of $\boldsymbol{Z}_k$ and $\boldsymbol{X}_k$ coincide:

$$\boldsymbol{X}_k\boldsymbol{X}_k^t = \boldsymbol{Z}_k\boldsymbol{V}_k^t\boldsymbol{V}_k\boldsymbol{Z}_k^t = \boldsymbol{Z}_k\boldsymbol{Z}_k^t,$$

in which we made use of the orthonormality of the columns of $\boldsymbol{V}_k$.

Thus $\boldsymbol{D}_{Zk} = \boldsymbol{D}(\boldsymbol{X}_k)$ is the best approximation of $\boldsymbol{D}_X = \boldsymbol{D}(\boldsymbol{X})$:

- the points in $\boldsymbol{X}_k$ are p-dimensional and cannot be used directly to make a 2 or 3 dimensional graph. However, we know that $\boldsymbol{Z}_k = \boldsymbol{X}_k\boldsymbol{V}_k$, and the rows of $\boldsymbol{Z}_k$ are k-dimensional (e.g. $k = 2$ or $k = 3$). Note that also $\boldsymbol{Z}_k = \boldsymbol{U}_k\boldsymbol{D}_k$.
- The solution (graph in low k dimensions) requires $\boldsymbol{Z}_k$, which comes from the SVD of $\boldsymbol{X}$.

Hence, the data matrix $\boldsymbol{X}$ must be known!

We now look for a solution that does not require $\boldsymbol{X}$.

Let $\boldsymbol{D}_X$ be a given squared distance matrix (no knowledge of the p-dimensional $\boldsymbol{X}$).

Suppose we want to find the distance matrix

$$\boldsymbol{D}_{Zk} = \text{ArgMin}_{D_A:\text{rank}(D_A)=k} \|\boldsymbol{D}_X - \boldsymbol{D}_A\|_F^2.$$

By multiplying the distance matrices with $\boldsymbol{H}$ (left and right) this becomes

$$\boldsymbol{G}_k = \text{ArgMin}_{G_A:\text{rank}(G_A)=k} \|\boldsymbol{G}_X - \boldsymbol{G}_A\|_F^2.$$

This has the same form as the original SVD criterion and hence $\boldsymbol{G}_k$ is the truncated SVD of $\boldsymbol{G}_X = -\frac{1}{2}\boldsymbol{H}\boldsymbol{D}_X\boldsymbol{H}$.

Since $\boldsymbol{G}_X = -\frac{1}{2}\boldsymbol{H}\boldsymbol{D}_X\boldsymbol{H}$ is a symmetric matrix, its SVD is of the form

$$\boldsymbol{G}_X = \boldsymbol{U}\boldsymbol{S}\boldsymbol{U}^t,$$

with $\boldsymbol{U}$ the matrix with the singular vectors, and $\boldsymbol{S}$ a diagonal matrix with the singular values of $\boldsymbol{G}_X$.

The best rank k approximation is thus given by the truncated SVD

$$\boldsymbol{G}_k = \boldsymbol{U}_k\boldsymbol{S}_k\boldsymbol{U}_k^t.$$

Remember $\boldsymbol{G}_k = \boldsymbol{Z}_k\boldsymbol{Z}_k^t$ is the Gram matrix of some $n \times k$ matrix $\boldsymbol{Z}_k$. If we set

$$\boldsymbol{Z}_k = \boldsymbol{U}_k\boldsymbol{S}_k^{1/2},$$

we get again $\boldsymbol{G}_k = \boldsymbol{U}_k\boldsymbol{S}_k\boldsymbol{U}_k^t$. Hence, $\boldsymbol{Z}_k$ can be computed from only $\boldsymbol{D}_X$ only!

Finally note that, if $\boldsymbol{X}$ is known, the SVD $\boldsymbol{X} = \boldsymbol{U}\boldsymbol{D}\boldsymbol{V}^t$ gives $\boldsymbol{X}\boldsymbol{X}^t = \boldsymbol{U}\boldsymbol{D}^2\boldsymbol{U}^t$, from which we find $\boldsymbol{S} = \boldsymbol{D}^2$.

Hence, $\boldsymbol{Z}_k$ can be computed from either the SVD of $\boldsymbol{X}$ or from the SVD of $\boldsymbol{G}_X = -\frac{1}{2}\boldsymbol{H}\boldsymbol{D}_X\boldsymbol{H}$. The latter only requires knowledge of a squared distance matrix $\boldsymbol{D}_X$.

Since (1) the SVD of $\boldsymbol{X} = \boldsymbol{U}\boldsymbol{D}\boldsymbol{V}^t$ gives $\boldsymbol{G}_X = \boldsymbol{X}\boldsymbol{X}^t = \boldsymbol{U}\boldsymbol{D}^2\boldsymbol{U}^t$, and (2) the SVD of $\boldsymbol{G}_X = \boldsymbol{U}\boldsymbol{S}\boldsymbol{U}^t$, we find $\boldsymbol{S} = \boldsymbol{D}^2$. Thus the left-singular vectors of $\boldsymbol{X}$ and $\boldsymbol{G}_X$ coincide, and the singular values of the latter are the squares of the former.

2.4.3 Examples

Example: Australian cities

Given is a matrix with the distances (in km) between major Australian cities.

```
> DX<-read.csv("../Data/AussieDistancesKM.txt",
     sep=" ",header=TRUE)
> rownames(DX)<-colnames(DX)
> DX
        Adel  Alice  Bris  Cairns  Canb  Darwin  Melb  Perth   Syd
Adel       0   1533  2044    3143  1204    3042   728   2725  1427
Alice   1533      0  3100    2500  2680    1489  2270   3630  2850
Bris    2044   3100     0    1718  1268    3415  1669   4384  1010
Cairns  3143   2500  1718       0  2922    3100  3387   5954  2730
Canb    1204   2680  1268    2922     0    3917   647   3911   288
Darwin  3042   1489  3415    3100  3917       0  4045   4250  3991
Melb     728   2270  1669    3387   647    4045     0   3430   963
Perth   2725   3630  4384    5954  3911    4250  3430      0  4110
Syd     1427   2850  1010    2730   288    3991   963   4110     0
```

Example: Australian cities

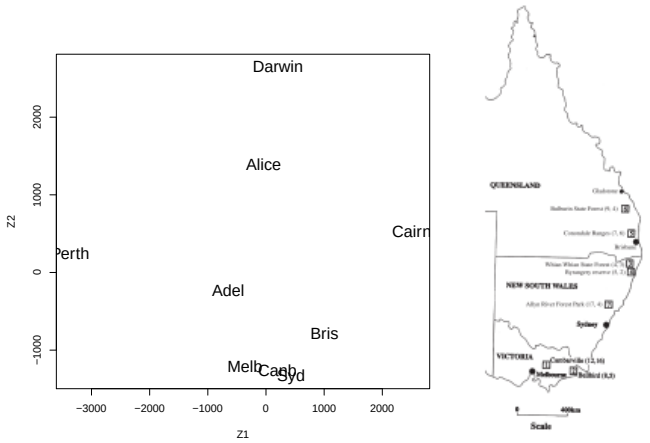
Next we compute the $\boldsymbol{G}_X$ matrix, its SVD and the $\boldsymbol{Z}_k$ for $k = 2$.

```
DX<-as.matrix(DX^2) # squared dist. matrix
n<-nrow(DX)
H<-diag(n)-1/n*matrix(1,ncol=n,nrow=n)
GX<--1/2*H%*%DX%*%H
GX.svd<-svd(GX)

k<-2
Uk<-GX.svd$u[,1:k]
Sk<-diag(GX.svd$d[1:k])
Zk<-Uk%*%sqrt(Sk)
rownames(Zk)=colnames(DX)

plot(Zk,type="n",xlab="Z1",ylab="Z2")
text(Zk,rownames(Zk),cex=1.5)
```

Example: Australian cities



The rows in $\boldsymbol{Z}_2$ (i.e. $\boldsymbol{Z}_k$ with $k = 2$) give a very good approximation of the reality. The reason why not an exact match is found:

- the distance matrix $\boldsymbol{D}_X$ gives distances along the roads.
- even if geodesic distances were given, then still a 2-dimensional graph would not exactly represent the original geodesic distances, because the latter refer to distances on a ball (the earth)
- the solution of a SVD is only unique up to some rotations of the column basis of $\boldsymbol{U}$ and $\boldsymbol{V}$.

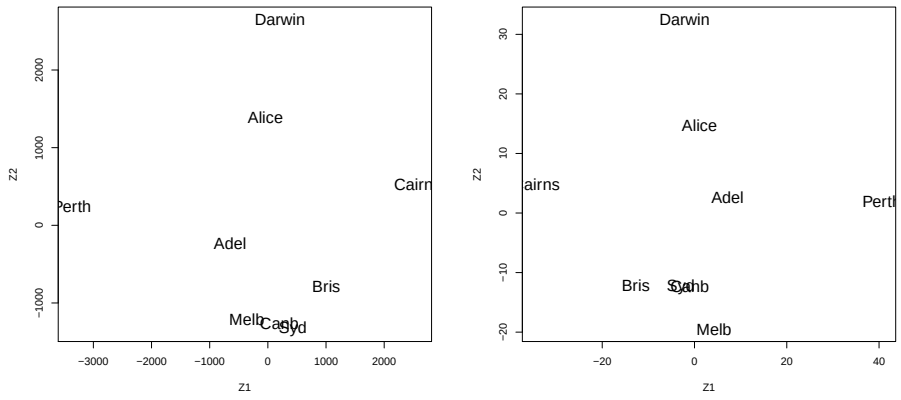
Example: Australian cities

Given is a matrix with the distances (in travelling time, hours) between major Australian cities.

```
> DX
        Adel  Alice  Bris  Cairns  Canb  Darwin  Melb  Perth  Syd
Adel       0     16    23      38    16      24     9     30   15
Alice     16      0    37      30    32      18    24     40   32
Bris      23     37     0      21    16      44    19     53   11
Cairns    38     30    21       0    35      37    40     75   32
Canb      16     32    16      35     0      43     7     43    3
Darwin    24     18    44      37    43       0    55     49   43
Melb       9     24    19      40     7      55     0     39    9
Perth     30     40    53      75    43      49    39      0   44
Syd       15     32    11      32     3      43     9     44    0
```

Example: Australian cities

MDS plot for distance in km (left) and distance in traveling time (right).



Example: UK consumption

The data set gives the average consumption of 17 types of food ($p = 17$) for the four UK countries ($n = 4$). Can we depict the 17-dimensional data in a 2-dimensional plane (i.e. $k = 2$)?

```
> UKCon<-read.csv("../Data/UKConsumption.txt",sep=" ",header=T)
> t(UKCon)
             [,1]      [,2]         [,3]       [,4]
Country      "England" "N.Ireland" "Scotland" "Wales"
AlcDrinks    "375"     "135"      "458"      "475"
Beverages    "57"      "47"       "53"       "73"
CarcaseMeat  "245"     "267"      "242"      "227"
Cereals      "1472"    "1494"     "1462"     "1582"
Cheese       "105"     " 66"      "103"      "103"
Confectionery "54"     "41"       "62"       "64"
FatsOils     "193"     "209"      "184"      "235"
Fish         "147"     " 93"      "122"      "160"
FreshFruit   "1102"    " 674"     " 957"     "1137"
FreshPot     " 720"    "1033"     " 566"     " 874"
FreshVeg     "253"     "143"      "171"      "265"
OtherMeat    "685"     "586"      "750"      "803"
OtherVeg     "488"     "355"      "418"      "570"
ProcPot      "198"     "187"      "220"      "203"
ProcVeg      "360"     "334"      "337"      "365"
SoftDrinks   "1374"    "1506"     "1572"     "1256"
Sugars       "156"     "139"      "139"      "175"
```

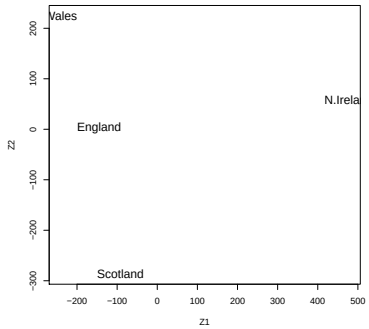
This is an example where we start from a $p = 17$ dimensional data set $\boldsymbol{X}$, and we want to see which UK countries are similar w.r.t. their food consumption.

Note that the data is $p = 17$ dimensional, but we have only data on 4 countries. The matrix $\boldsymbol{X}$ is thus a 4×17 matrix and its rank cannot be larger than $\min(4, 17) = 4$. Thus the 4 points in the 17-dimensional space actually live in a 4 dimensional subspace.

Example: UK consumption

```
> X<-UKCon[,-1]
> n<-nrow(X)
> H<-diag(n)-1/n*matrix(1,ncol=n,nrow=n)
> X[,]<-H%*%as.matrix(X)
> X.svd<-svd(X)
> k<-2
> Uk<-X.svd$u[,1:k]
> Dk<-diag(X.svd$d[1:k])
> Zk<-Uk%*%Dk
> rownames(Zk)<-UKCon[,1]
> Zk
                [,1]       [,2]
England   -145.07787    2.56379
N.Ireland  477.30415   59.12818
Scotland   -91.57278 -286.39897
Wales     -240.65350  224.70700
> plot(Zk,type="n",xlab="Z1",ylab="Z2",
    xlim=c(-2800,-2550))
> text(Zk,rownames(Zk),cex=1.3)
```

Example: UK consumption



A few questions remain:

- How well can the 17-dimensional data be represented in a 2-dimensional subspace?
- How can we interpret the distances (differences) between the data points in terms of the original 17 variables?

The graph suggests that Wales and England are quite similar in terms of food consumption, and North Ireland seem to have a very different food consumption pattern. At the moment we cannot say more. We first need to get an interpretation for the two axes so that we can express similarities and dissimilarities between the four UK countries in terms of the 17 food products.

2.5 The Biplot

The biplot is a single 2-dimensional graph which displays the information in both $\boldsymbol{Z}_2 = \boldsymbol{U}_2\boldsymbol{D}_2$ and $\boldsymbol{V}_2$. The plot of $\boldsymbol{Z}_2$ has been discussed previously (e.g. best approximation of distances). The name "bi"plot refers to the plotting of two parts of the SVD in a single graph.

The Biplot

As discussed previously, each k-dimensional row in $\boldsymbol{Z}_k$ is the result of an orthogonal projection of the original row in $\boldsymbol{X}$ onto the space spanned by the k columns in $\boldsymbol{V}_k$,

$$\boldsymbol{z}_{ki}^t = \boldsymbol{x}_i^t \boldsymbol{V}_k = \left(\boldsymbol{x}_i^t \boldsymbol{v}_1, \boldsymbol{x}_i^t \boldsymbol{v}_2, \ldots, \boldsymbol{x}_i^t \boldsymbol{v}_k\right).$$

From the inverse transformation, $\boldsymbol{X}_k = \boldsymbol{Z}_k \boldsymbol{V}_k^t$, we see that the ith row of $\boldsymbol{X}_k$ is the result of the orthogonal projection of the ith row of $\boldsymbol{Z}_k$ onto the subspace spanned by the p rows of $\boldsymbol{V}_k$:

$$\boldsymbol{x}_{ki}^t = \boldsymbol{z}_{ki}^t \boldsymbol{V}_k^t = \boldsymbol{z}_{ki}^t \begin{pmatrix} \tilde{\boldsymbol{v}}_{k1} & \cdots & \tilde{\boldsymbol{v}}_{kp} \end{pmatrix} = \left(\boldsymbol{z}_{ki}^t \tilde{\boldsymbol{v}}_{k1}, \boldsymbol{z}_{ki}^t \tilde{\boldsymbol{v}}_{k2}, \ldots, \boldsymbol{z}_{ki}^t \tilde{\boldsymbol{v}}_{kp}\right),$$

where $\tilde{\boldsymbol{v}}_{kj}$ is the jth column of $\boldsymbol{V}_k^t$, or $\tilde{\boldsymbol{v}}_{kj}^t$ is the jth row of $\boldsymbol{V}_k$ (k-dimensional).

$$\boldsymbol{x}_{ki}^t = \boldsymbol{z}_{ki}^t \boldsymbol{V}_k^t = \left(\boldsymbol{z}_{ki}^t \tilde{\boldsymbol{v}}_{k1}, \boldsymbol{z}_{ki}^t \tilde{\boldsymbol{v}}_{k2}, \ldots, \boldsymbol{z}_{ki}^t \tilde{\boldsymbol{v}}_{kp}\right),$$

Hence, the single data point x_{kij} is given by $\boldsymbol{z}_{ki}^t \tilde{\boldsymbol{v}}_{kj}$.

If we plot the p vectors $\tilde{\boldsymbol{v}}_{kj}$ ($j = 1, \ldots, p$), then x_{kij} can be read from the graph by projecting $\boldsymbol{z}_{ki}$ orthogonally onto $\tilde{\boldsymbol{v}}_{kj}$.

If $k = r$, the original observation x_{ij} can be exactly reconstructed by projecting $\boldsymbol{z}_i$ onto $\tilde{\boldsymbol{v}}_{kj}$.

$$\boldsymbol{x}_i^t = \boldsymbol{z}_i^t \boldsymbol{V}^t = (\boldsymbol{z}_i^t \tilde{v}_{r1}, \boldsymbol{z}_i^t \tilde{v}_{r2}, \ldots, \boldsymbol{z}_i^t \tilde{v}_{rp}).$$

For $k < r$, the reconstruction is only an approximation.

Example: UK consumption

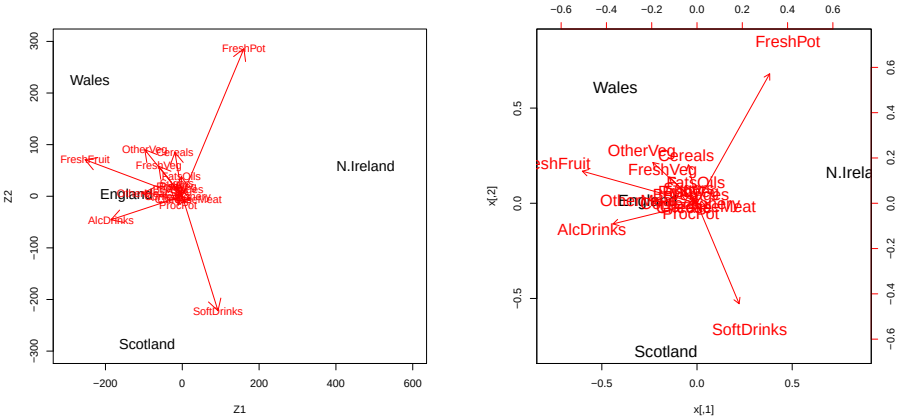
```
Vk<-X.svd$v[,1:k]
Vk<-as.data.frame(Vk)
rownames(Vk)<-names(UKCon[,-1])

plot(Zk,type="n",xlab="Z1",ylab="Z2",xlim=c(-300,600),
     ylim=c(-300,300))
text(Zk,rownames(Zk),cex=1.3)
gamma<-400 # rescaling to get better visualisation
for(i in 1:17) {
  arrows(0,0,gamma*Vk[i,1],gamma*Vk[i,2],
       length = 0.2,col=2)
  text(gamma*Vk[i,1],gamma*Vk[i,2],rownames(Vk)[i],
       col=2)
}
biplot(Uk,Vk,xlabs=rownames(Zk),cex=1.5,
     ylabs=rownames(Vk))
```

The R code shows two ways of constructing a biplot: the first method starts from the SVD of $\boldsymbol{X}$ and plots the p vectors $\tilde{\boldsymbol{v}}_{2j}$ as arrows. Note that here a scaling factor gamma is used. This is used to give the arrows a convenient length in the graph (not too small, not too large); it does not affect the interpretation. The second way of obtaining the biplot is simply by using the R built-in function biplot.

Example: UK consumption

Biplots



A few remarks/conclusions about the biplot:

- the origin corresponds to the sample average of the 17-dimensional observations in the data matrix
- England is close to the origin and thus one could say that the food consumption in England is as the average in the UK
- projecting North Ireland orthogonally onto the vector $\tilde{\boldsymbol{v}}_{2j}$ of fresh potatoes, we get a large and positive x_{2ij}. Hence, people in N. Ireland tend to eat more fresh potatoes than on average in the UK
- projecting North Ireland orthogonally onto the vector $\tilde{\boldsymbol{v}}_{2j}$ of soft drinks, we get small but positive x_{2ij}. Hence, people in N. Ireland tend to drink a bit more soft drinks than on average in the UK
- projecting North Ireland orthogonally onto the vector $\tilde{\boldsymbol{v}}_{2j}$ of alcoholic drinks, we get a large and negative x_{2ij}. Hence, people in N. Ireland tend to drink less alcoholic drinks than on average in the UK

Let's move to a deeper interpretation.

From $\boldsymbol{x}_{ki}^t = \boldsymbol{z}_{ki}^t \boldsymbol{V}_k^t$ we learnt that the original data can be (approximately) reconstructed from the $\boldsymbol{z}_{ki}$.

We now show how the dimensions of $\boldsymbol{Z}_k$ (columns or variables) can be interpreted. Remember,

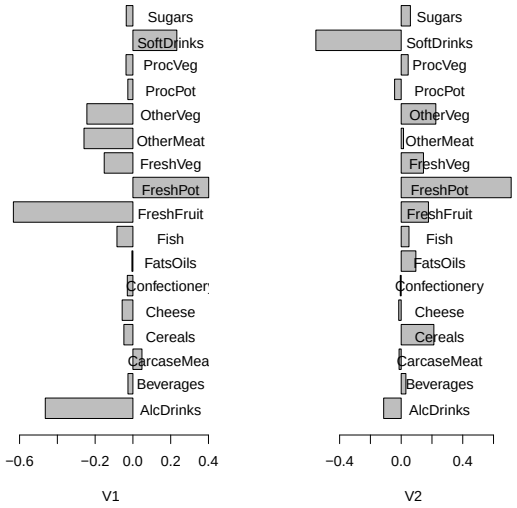
$$\boldsymbol{z}_{ki}^t = \boldsymbol{x}_i^t \boldsymbol{V}_k = \left(\boldsymbol{x}_i^t \boldsymbol{v}_1, \boldsymbol{x}_i^t \boldsymbol{v}_2, \ldots, \boldsymbol{x}_i^t \boldsymbol{v}_k\right).$$

An individual element of $\boldsymbol{Z}_k$ can be written as

$$z_{kij} = \boldsymbol{x}_i^t \boldsymbol{v}_j,$$

which is a linear combination of the observations on the p variables in $\boldsymbol{x}_i$, with coefficients given by the p elements in $\boldsymbol{v}_j$. Understanding the elements in $\boldsymbol{v}_j$ will give us insight into the interpretation of the jth dimension of $\boldsymbol{Z}_k$.

Example: UK consumption



In giving an interpretation to the elements in $\boldsymbol{v}_1$ and $\boldsymbol{v}_2$, we ignore the elements close to zero (this is a subjective decision; later we see more objective methods).

For $\boldsymbol{v}_1$ (1st dimension of $\boldsymbol{Z}_2$):

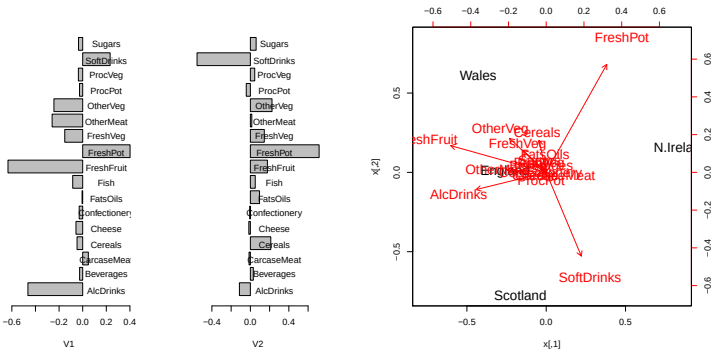
- contrast of soft drinks and fresh potatoes versus fresh fruit and alcoholic drinks
- a large value of z_{i1} can result from eating many fresh potatoes and drinking a lot of soft drinks, but eating only few fresh fruit and drinking not much alcoholic drinks
- a small value of z_{i1} can result from eating few fresh potatoes and drinking only few soft drinks, but eating a lot of fresh fruit and drinking much alcoholic drinks

For $\boldsymbol{v}_2$ (2nd dimension of $\boldsymbol{Z}_2$):

- contrast of soft drinks versus fresh potatoes
- a large value of z_{i2} can result from eating many fresh potatoes, but drinking not much soft drinks
- a small value of z_{i2} can result from eating few fresh potatoes, but drinking much soft drinks .

Example: UK consumption

The elements in $\boldsymbol{v}_1$ and $\boldsymbol{v}_2$ are also shown in the biplot.



From the graph we see e.g. that N. Ireland has a high score for the first dimension. Now that we can give an interpretation to the dimension, we conclude that in Northern Ireland people eat relatively much fresh potatoes and drink many soft drinks, but they do not drink much alcoholic drinks and eat not much fresh fruit. We had already come to these findings by projecting N. Ireland on the vectors of these four food products. This comes to no surprise: both interpretations arise from the same data set and its SVD, and so no contradictory results should arise. Other conclusions can be found in a similar fashion.

We have derived the interpretation of the first dimension from the barplot of the elements of $\boldsymbol{v}_1$. However, there is actually no need to make a separate plot to read the elements of $\boldsymbol{v}_1$; they can also be read from projecting the vectors in the biplot onto the first dimension (i.e. basis vector of the first dimension). For example, fresh potatoes is in the 8th column of $\boldsymbol{X}$ and thus in the biplot its vector is $\tilde{\boldsymbol{v}}_{28}$; it is a 2-dimensional vector (2-dimensional biplot is shown). In the space of the biplot (i.e. the column space of $\boldsymbol{Z}_2$), the first basis vector is given by $(1, 0)$. Projecting $\tilde{\boldsymbol{v}}_{28}^t$, which is the eight row of $\boldsymbol{V}_2$, orthogonally onto $(1, 0)$ gives

$$\tilde{\boldsymbol{v}}_{28}^t \begin{pmatrix} 1 \\ 0 \end{pmatrix} = (v_{81}, v_{82}) \begin{pmatrix} 1 \\ 0 \end{pmatrix} = v_{81}$$

which is the eight element of the first right-singular vector of $\boldsymbol{X}$, which is thus the bar of fresh potatoes in the barplot of the first dimension.

2.6 Principal Component Analysis

2.6.1 Introduction

Principal Component Analysis (PCA)

- PCA is basically a SVD
- PCA adds another layer of interpretation
- PCA comes with its own terminology

Principal Component Analysis (PCA) is one of the most common methods used to explore a multivariate data set. It is implemented in most statistical software. In this section we will first introduce the PCA in a traditional way (as you may find in most textbooks), and next we show how PCA is actually a SVD with an extra layer of interpretation.

For a given centered data matrix $\boldsymbol{X}$ the $p \times p$ covariance matrix can be estimated by

$$\boldsymbol{\Sigma}_X = \frac{1}{n-1}\boldsymbol{X}^t\boldsymbol{X} = \frac{1}{n-1}\sum_{i=1}^{n}\boldsymbol{x}_i\boldsymbol{x}_i^t,$$

i.e. the (i,j)th element is given by (column means are zero)

$$\frac{1}{n-1}\sum_{m=1}^{n}x_{mi}x_{mj} = \frac{1}{n-1}\sum_{m=1}^{n}(x_{mi}-\bar{x}_i)(x_{mj}-\bar{x}_j).$$

In the remainder of this section I will often write the factor $1/(n-1)$, but perhaps that I forget it sometimes. For what follows the factor is not important. It is only a proportionality factor and it will not affect the interpretations.

2.6.2 Traditional Formulation of PCA

PCA is usually introduced as follows. Let

$$y_i = \boldsymbol{x}_i^t \boldsymbol{a}$$

with $\boldsymbol{a}$ a p-vector of constants. Hence y_i is a linear combination (or transformation) of $\boldsymbol{x}_i$.

PCA aims at finding $\boldsymbol{a}$ such that the variance among the n y_i's is maximal, with

$$\begin{aligned}
\text{Var}\{y\} &= \frac{1}{n-1}\sum_{i=1}^{n}(y_i - \bar{y})^2 = \frac{1}{n-1}\sum_{i=1}^{n} y_i^2 \\
&= \frac{1}{n-1}\sum_{i=1}^{n}(\boldsymbol{x}_i^t \boldsymbol{a})^2 = \frac{1}{n-1}\boldsymbol{a}^t \left(\sum_{i=1}^{n} \boldsymbol{x}_i \boldsymbol{x}_i^t\right) \boldsymbol{a} \\
&= \frac{1}{n-1}\boldsymbol{a}^t \boldsymbol{X}^t \boldsymbol{X} \boldsymbol{a} = \boldsymbol{a}^t \boldsymbol{\Sigma}_X \boldsymbol{a}
\end{aligned}$$

in which $\boldsymbol{\Sigma}_X = \frac{1}{n-1}\boldsymbol{X}^t\boldsymbol{X}$ is the sample covariance matrix of $\boldsymbol{X}$.

We may have made an abuse of notation. We use Var $\{y\}$ to denote the sample variance of the y_i, whereas the notation is conventionally used for the population variance. Similarly we use $\boldsymbol{\Sigma}$ to denote the sample covariance matrix of the data $\boldsymbol{X}$, whereas it is typically used as notation for a population covariance matrix. Mixing up notation of estimates and parameters is allowed here because we only use the methods for data exploration. Thus we do not do inference (i.e. generalising from sample to larger population).

Finding $\boldsymbol{a}$ that maximises

$$\text{Var}\,\{y\} = \boldsymbol{a}^t \boldsymbol{\Sigma}_X \boldsymbol{a}$$

has a trivial solution (set all elements of $\boldsymbol{a}$ equal to ∞). To avoid the trivial solution, the solution must satisfy a restriction, e.g.

$$\|\boldsymbol{a}\|^2 = \boldsymbol{a}^t \boldsymbol{a} = 1.$$

The solution of the constrained maximisation problem may be formulated as

$$\boldsymbol{a} = \text{ArgMax}_{b:\|b\|^2=1} \boldsymbol{b}^t \boldsymbol{\Sigma}_X \boldsymbol{b}.$$

By introducing a Lagrange multiplier λ, we get an unconstrained maximisation problem,

$$\boldsymbol{a} = \text{ArgMax}_b \left(\boldsymbol{b}^t \boldsymbol{\Sigma}_X \boldsymbol{b} - \lambda(\boldsymbol{b}^t \boldsymbol{b} - 1) \right).$$

If the constraint is linear, then the constrained optimisation problem (here: maximisation) may be replaced by an unconstrained optimisation problem of the same criterion but with an "penalty term" added. This method is due to Lagrange. The penalty term vanishes when the constraint is satisfied.

Note that if the constraint is satisfied, $\boldsymbol{b}^t \boldsymbol{b} = \|\boldsymbol{b}\|^2 = 1$ and thus $\boldsymbol{b}^t \boldsymbol{b} - 1 = 0$ and the "penalty" term in the unconstrained criterion vanishes.

The Lagrange multiplier λ is to be considered as an extra parameter that may have to be estimated from the data.

The solution of

$$\boldsymbol{a} = \text{ArgMax}_b \left(\boldsymbol{b}^t \boldsymbol{\Sigma}_X \boldsymbol{b} - \lambda(\boldsymbol{b}^t \boldsymbol{b} - 1) \right)$$

is obtained by differentiating $\boldsymbol{b}^t \boldsymbol{\Sigma}_X \boldsymbol{b} - \lambda(\boldsymbol{b}^t \boldsymbol{b} - 1)$ w.r.t. $\boldsymbol{b}$, equating it to zero and solving for $\boldsymbol{b}$.

$$\begin{aligned} \frac{\partial}{\partial \boldsymbol{b}} \left(\boldsymbol{b}^t \boldsymbol{\Sigma}_X \boldsymbol{b} - \lambda(\boldsymbol{b}^t \boldsymbol{b} - 1) \right) &= 0 \\ 2\boldsymbol{\Sigma}_X \boldsymbol{b} - 2\lambda \boldsymbol{b} &= 0. \end{aligned}$$

Hence, we need the solution of

$$\boldsymbol{\Sigma}_X \boldsymbol{b} = \lambda \boldsymbol{b}.$$

This equation has r solutions:

- $\boldsymbol{b} = \boldsymbol{e}_j$: the jth eigenvector of $\boldsymbol{\Sigma}_X$
- $\lambda = \lambda_j$: the jth eigenvalue of $\boldsymbol{\Sigma}_X$.

The eigenvectors are orthonormal, i.e. $\boldsymbol{e}_i^t \boldsymbol{e}_j = \delta_{ij}$.

The equation

$$\boldsymbol{\Sigma}_X \boldsymbol{b} = \lambda \boldsymbol{b}$$

expresses a system of linear equations, and it is very well known in matrix algebra.

Mind the difference in terminology between eigenvectors and eigenvalues in this context, and singular vectors and singular values in the SVD context. Later we will see that they are related to one another.

Consider the following calculations, with $\boldsymbol{b} = \boldsymbol{e}_j$, the $j$the eigenvector of $\boldsymbol{\Sigma}_X$.

$$\begin{aligned} \text{Var}\{y\} &= \boldsymbol{e}_j^t \boldsymbol{\Sigma}_X \boldsymbol{e}_j \\ &= \boldsymbol{e}_j^t (\boldsymbol{\Sigma}_X \boldsymbol{e}_j) \\ &= \boldsymbol{e}_j^t (\lambda_j \boldsymbol{e}_j) \\ &= \lambda_j \boldsymbol{e}_j^t \boldsymbol{e}_j \\ &= \lambda_j. \end{aligned}$$

By convention the eigenvectors/eigenvalues are ordered so that

$$\lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_r.$$

Hence, the first eigenvector $\boldsymbol{e}_1$ gives the largest variance λ_1. Hence, this is the solution we were looking for.

Later we look at how the other solutions can help us.

We now change from the y-notation to the z-notation (first hint that PCA and SVD are related).

The observations of the first principal component (PC) are then given by

$$z_{i1} = \boldsymbol{x}_i^t \boldsymbol{e}_1.$$

In PCA terminology they are referred to as the scores of the first PC. The elements in the eigenvector $\boldsymbol{e}_i$ that make up the transformation are known as the loadings of the first PC.

The first PC is thus a new variable (construct) that has the largest variance among all linear transformations of the original variables.

Variability between observations is considered as informative to understand differences between the observations.

The equation

$$\boldsymbol{\Sigma}_X \boldsymbol{b} = \lambda \boldsymbol{b}$$

has $r = \text{rank}(\boldsymbol{\Sigma}_X)$ solutions. The eigenvectors are orthonormal, i.e. $\boldsymbol{e}_i^t \boldsymbol{e}_j = \delta_{ij}$.

Hence (for $i \neq j$)

$$\text{Cov}\left\{\boldsymbol{x}^t \boldsymbol{e}_i, \boldsymbol{x}^t \boldsymbol{e}_j\right\} = \boldsymbol{e}_i^t \text{Var}\left\{\boldsymbol{x}\right\} \boldsymbol{e}_j = \boldsymbol{e}_i^t \boldsymbol{\Sigma}_X \boldsymbol{e}_j = \boldsymbol{e}_i^t \left(\lambda_j \boldsymbol{e}_j\right) = \lambda_j \boldsymbol{e}_i^t \boldsymbol{e}_j = 0$$

If $z_j = \boldsymbol{x}_i^t \boldsymbol{e}_j$ denotes the jth PC, then

$$\text{Cov}\left\{z_i, z_j\right\} = \text{Cov}\left\{\boldsymbol{x}^t \boldsymbol{e}_i, \boldsymbol{x}^t \boldsymbol{e}_j\right\} = 0 \text{ if } i \neq j$$

$$\text{Var}\left\{z_j\right\} = \lambda_j.$$

We say that the jth PC maximises the variance among all linear transformations such that it is uncorrelated with the previous PCs.

Interpretation of PCA

A PCA is a transformation of the original p variables to r PCs such that

- the first PC has largest variance, equal to first eigenvalue of $\mathbf{\Sigma}_X$
- the next PCs have decreasing variances (decreasing information content)
- all PCs are mutually uncorrelated (no information-overlap).

You may notice that we suggest that variance and information are two of the same kind.

Consider the very extreme situation in which all observations coincide in the same point. So variability between observations is zero. In this setting one cannot learn to understand differences between observations.

Of course, if all observations on the original p-dimensional space concentrate into one single point, then there is nothing to learn about the differences between the observations (thus no information). However, here we are talking about a situation in which the n observations in the p-dimensional space are distinct, but by choosing a transformation to a reduced dimensional space we have the freedom to choose the transformation such that we can maximise variability. In this reasoning one often relates "variability" to "information". The more variability among the observations, the more information (i.e. the more you can learn from them).

2.6.3 Relation between PCA and SVD

PCA and SVD

The r solutions of the eigenvectors/eigenvalues of $\boldsymbol{\Sigma}_X$ may simultaneously be written as

$$\boldsymbol{\Sigma}_X \boldsymbol{E} = \boldsymbol{E}\boldsymbol{L}$$

with

- $\boldsymbol{E}$ the $p \times r$ matrix with columns $\boldsymbol{e}_j$. By the orthonormality of the eigenvectors, we get $\boldsymbol{E}\boldsymbol{E}^t = \boldsymbol{E}^t\boldsymbol{E} = \boldsymbol{I}$
- $\boldsymbol{L}$ the $r \times r$ diagonal matrix with elements λ_j.

Right-multiplying both sides of $\boldsymbol{\Sigma}_X \boldsymbol{E} = \boldsymbol{E}\boldsymbol{L}$ with $\boldsymbol{E}^t$, we find

$$\boldsymbol{\Sigma}_X = \boldsymbol{E}\boldsymbol{L}\boldsymbol{E}^t.$$

On the other hand, the SVD of the symmetric matrix $\boldsymbol{\Sigma}_X$ gives

$$\boldsymbol{\Sigma}_X = \boldsymbol{V}\boldsymbol{S}\boldsymbol{V}^t.$$

Hence,

- the eigenvectors of $\boldsymbol{\Sigma}_X$ coincide with the singular vectors of $\boldsymbol{\Sigma}_X$
- the eigenvalues of $\boldsymbol{\Sigma}_X$ coincide with the singular values of $\boldsymbol{\Sigma}_X$.

Thus, although the loadings that maximise the variance of the first PC were first derived as the first eigenvector of $\boldsymbol{\Sigma}_X$, we now have learnt that the solution may also be obtained by the first right-singular vector of the SVD of $\boldsymbol{\Sigma}_X$.

Another way of looking at the SVD solution to PCA.

Consider the following problem: for a given covariance matrix $\boldsymbol{\Sigma}_X$, find for $k \leq r$

$$\boldsymbol{\Sigma}_k = \text{ArgMin}_{\Sigma:\text{rank}(\Sigma)=k} \|\boldsymbol{\Sigma}_X - \boldsymbol{\Sigma}\|_F^2.$$

This minimisation problem is the same as in the SVD, but now applied to a symmetric covariance matrix.

Hence, $\boldsymbol{\Sigma}_k = \boldsymbol{V}_k \boldsymbol{S}_k \boldsymbol{V}_k^t$ (truncated SVD of symmetric matrix), with

- $\boldsymbol{V}_k$ the matrix with columns equal to the first k singular vectors of $\boldsymbol{\Sigma}_X$
- $\boldsymbol{S}_k$ the diagonal matrix with the first k singular values of $\boldsymbol{\Sigma}_X$.

Up to here we have shown that the eigenvectors and eigenvalues of $\boldsymbol{\Sigma}_X$ coincide with the right-singular vectors and singular values of $\boldsymbol{\Sigma}_X$. Both solutions are based on the covariance matrix $\boldsymbol{\Sigma}_X$, but they give us a different way of looking at the problem: maximising variance in the dimension-reduced PC-space, and best approximation of the original covariance matrix in a k-dimensional subspace.

We go one step further and relate the SVD of $\boldsymbol{\Sigma}_X$ to the SVD of the data matrix $\boldsymbol{X}$.

If $\boldsymbol{X}$ is observed, then

$$\boldsymbol{\Sigma}_X = \frac{1}{n-1}\boldsymbol{X}^t\boldsymbol{X} = \frac{1}{n-1}(\boldsymbol{U}\boldsymbol{D}\boldsymbol{V}^t)^t(\boldsymbol{U}\boldsymbol{D}\boldsymbol{V}^t) = \frac{1}{n-1}\boldsymbol{V}\boldsymbol{D}^2\boldsymbol{V}^t.$$

Hence, up to a factor $n-1$,

- the right-singular vectors of $\boldsymbol{X}$ coincide with the singular vectors (eigenvectors) of $\boldsymbol{\Sigma}_X$
- the singular values of $\boldsymbol{X}$ are the square roots of the singular values (eigenvalues) of $\boldsymbol{\Sigma}_X$. $(\lambda_j = \delta_j^2)$

The loadings of the PCs can thus also be obtained from the right-singular vectors of the SVD of $\boldsymbol{X}$!

Remember that the columns of $\boldsymbol{V}_k$ also span the MDS space, and we have used the elements of $\boldsymbol{V}$ to interpret the axes of the biplot so that a better interpretation of the MDS could be obtained.

MDS aimed at an optimal conservation of the distances between the original p-dimensional observations after projection onto a k-dimensional subspace. Now that we know that MDS-projection is the same as the PCA-projection, we get a second interpretation of the biplot. This second interpretation will become more clear after the next slide.

2.6.4 Conservation of Variance

Conservation of Variance

- The total variance in a data set $\boldsymbol{X}$ is given by the sum of the variances of the variables,
$$\sigma^2_{\text{tot}} = \sum_{j=1}^{p} \text{Var}\{X_j\} = \text{trace}(\boldsymbol{\Sigma}_X)$$
- For a symmetric matrix it holds that
$$\text{trace}(\boldsymbol{\Sigma}_X) = \sum_{j=1}^{r} \lambda_r.$$
- Since $\lambda_r = \text{Var}\{Z_j\}$, we find
$$\sigma^2_{\text{tot}} = \sum_{j=1}^{p} \text{Var}\{X_j\} = \sum_{j=1}^{r} \text{Var}\{Z_j\}.$$

Thus no information is lost by the PCA transformation from the original p-dimensional space to the p-dimensional PCA space.

If we assume that variance represents information, then we conclude that the PCA transformation from the original p-dimensional space (column space of $\boldsymbol{X}$) to the p-dimensional PC space (column space of $\boldsymbol{Z} = \boldsymbol{XV}$) gives no loss in information. However, this is a trivial statement, because the transformation $\boldsymbol{Z} = \boldsymbol{XV}$ is simply an orthonormal transformation of the basis of the original space (i.e. no projection onto a lower dimensional space). In practice, the PCA is used to accomplish a dimension reduction to a $k < r$ dimensional space. From

$$\sigma^2_{\text{tot}} = \sum_{j=1}^{p} \text{Var}\{X_j\} = \sum_{j=1}^{r} \lambda_j = \sum_{j=1}^{r} \text{Var}\{Z_j\}$$

we see that information is lost when only the first k dimensions are retained. Fortunately the PCs are ordered so that the first PCs contain the most information (largest variance), and thus it is hoped that with $k = 2$ or $k = 3$ most of the information is captured. We conclude that in this case the PCA has resulted in a dimension reduction that shows most of the information that was present in the original data set.

2.6.5 Choosing the number of dimensions

How many dimensions are needed?

For PCA the reasoning is usually based on the relative variance of a PC,

$$\frac{\text{Var}\,\{Z_j\}}{\sigma^2_{\text{tot}}} = \frac{\lambda_j}{\sum_{i=1}^r \lambda_i} = \frac{\delta_j^2}{\sum_{i=1}^r \delta_i^2}.$$

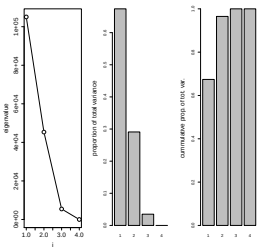
If the first k PCs are selected for further use, then they represent

$$100 \times \frac{\sum_{j=1}^k \lambda_j}{\sum_{i=1}^r \lambda_i}\% = 100 \times \frac{\sum_{j=1}^k \delta_j^2}{\sum_{i=1}^r \delta_i^2}\%$$

of the total variance (or information) of the original data set $\boldsymbol{X}$.

A scree plot is often used to look at the eigenvalues. Here it is shown for the UK consumption data.

```
totvar<-sum(X.svd$d^2)/(4-1)
plot(X.svd$d^2/(4-1),type="b",ylab="eigenvalue",
    xlab="j",cex=2,cex.axis=1.5,cex.lab=1.5)
barplot(X.svd$d^2/(4-1)/totvar,names.arg=1:4,
    ylab="proportion of total variance",cex.lab=1.5)
barplot(cumsum(X.svd$d^2/(4-1))/totvar,names.arg=1:4,
    ylab="cummulative prop. of tot. var.",cex.lab=1.5)
```



From these graphs we may conclude that with using $k = 2$ dimensions, more than 95% of the total variance is retained.

More generally, scree plots are also informative for the SVD. The motivation comes from

$$\text{Min}_{A:\text{rank}(A)=k}\|\boldsymbol{X}-\boldsymbol{A}\|_F^2 = \|\boldsymbol{X}-\boldsymbol{X}_k\|_F^2 = \sum_{j=k+1}^{r}\delta_j^2 = \sum_{j=k+1}^{r}\lambda_j.$$

This is known as the approximation error of the matrix $\boldsymbol{X}_k$.

Hence,

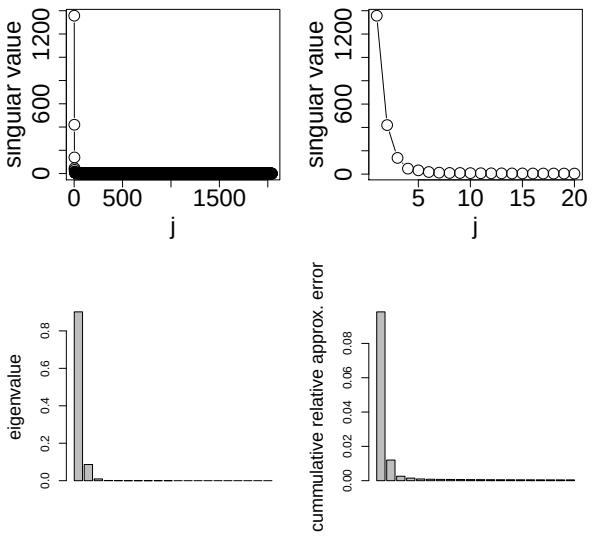
$$\frac{\sum_{j=1}^{k}\delta_j^2}{\sum_{j=1}^{r}\delta_j^2} = \frac{\sum_{j=1}^{k}\lambda_j}{\sum_{j=1}^{r}\lambda_j}$$

still makes sense as a relative quality measure for the SVD in general (including matrix approximation and MDS).

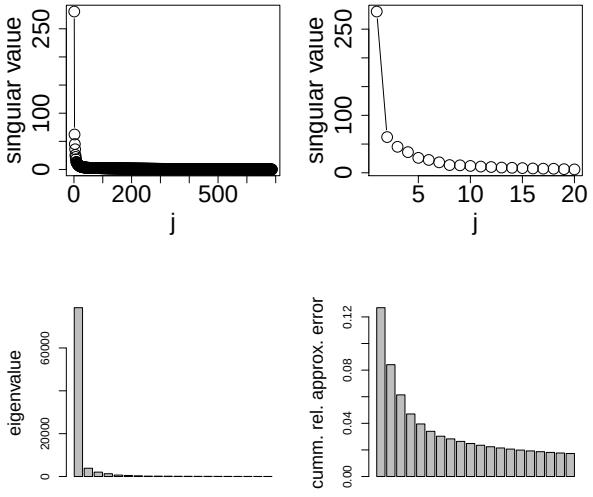
Some rules of thumb: select k such that

- at least 80% of the total variance is retained in the PC space, or, equivalently, at most 20% relative approximation error
- adding more dimensions does not add more information (in a relative sense); this can often be visually detected as the "knee" or "elbow" in the scree plot
- none of the dimensions has a variance smaller than the variance of one of the p original variables (this rule is typically only used when the variables are first standardised – see later).

Scree plots for the first Mondriaan picture.



Scree plots for the Mona Lisa.



The Mondriaan picture can be well approximated with only a few dimensions (e.g. $k = 3$). The more complex Mona Lisa needs many dimensions.

2.6.6 Covariance versus correlation matrix

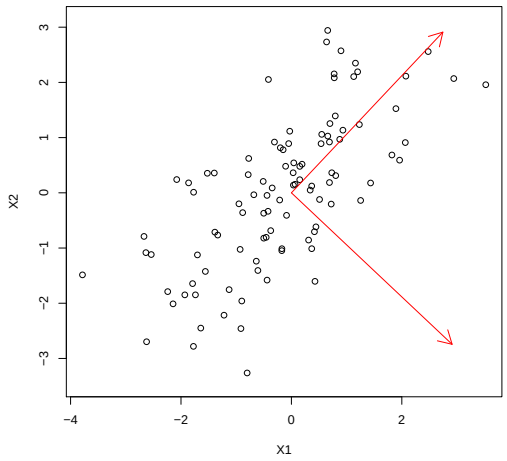
Direction of largest variability

Now that we know how the PCs are constructed, we can give an additional interpretation to the first eigenvector / right-singular vector of $\mathbf{\Sigma}_X$.

The first eigenvector of $\mathbf{\Sigma}_X$ is the direction in the original p-dimensional space of the largest variability.

The second eigenvector of $\mathbf{\Sigma}_X$ is the direction in the original p-dimensional space of the second largest variability, among all directions orthogonal to the first eigenvector

---



Covariance versus correlation matrix

So far we have worked with

- the column-centered data matrix $\boldsymbol{X}$
- the SVD of $\boldsymbol{X}$ and $\boldsymbol{\Sigma}_X \propto \boldsymbol{X}^t\boldsymbol{X}$ (the covariance matrix).

In some situations it is better to start from the standardised variables:

- substract from each element in $\boldsymbol{X}$ the column-mean
- divide each centered element in $\boldsymbol{X}$ by the column-specific standard deviation

Thus each element x_{ij} is replaced with

$$\frac{x_{ij} - \bar{x}_j}{s_j},$$

with x_j and s_j the column mean and column standard deviation.

In matrix notation, the standardisation, starting from the centered matrix $\boldsymbol{X}$ is computed as

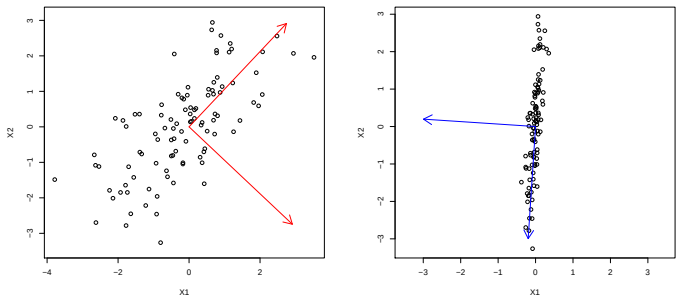
$$\boldsymbol{X}\boldsymbol{S}^{-1}$$

where $\boldsymbol{S}$ is a diagonal matrix with the column-specific standard deviations.

```
> var(X)
          [,1]      [,2]
[1,] 1.688727  1.221740
[2,] 1.221740  1.861574
> n<-nrow(X)
> H<-diag(n)-1/n*matrix(1,ncol=n,nrow=n)
> Xc<-H%*%X
> S<-diag(sqrt(diag(var(X))))
> Xs<-Xc%*%solve(S)
> var(Xs)
           [,1]       [,2]
[1,] 1.0000000  0.6890628
[2,] 0.6890628  1.0000000
> Xs<-scale(X,center=T,scale=T) # faster
```

After standardisation the variances are one.

Left: original matrix – Right: first column of $\boldsymbol{X}$ divided by 10 (e.g. moving from mm to cm).



Directions of maximal variability are affected by the units of the variables.

When to use correlation and when covariance?

- use correlation when columns of $\boldsymbol{X}$ are expressed in different units
- use covariance when columns of $\boldsymbol{X}$ are expressed in the same units.

There may be exceptions.

2.6.7 PCA and the multivariate normal distribution

PCA and the multivariate normal distribution

The density function of a multivariate normal distribution (MVN) is given by

$$f(\boldsymbol{x}) = (2\pi)^{-p/2}|\boldsymbol{\Sigma}|^{-1/2}\exp\left(-\frac{1}{2}(\boldsymbol{x}-\boldsymbol{\mu})^t\boldsymbol{\Sigma}^{-1}(\boldsymbol{x}-\boldsymbol{\mu})\right),$$

where

- $\boldsymbol{\mu}$ is the multivariate mean vector (p-dimensional). The jth element is $\mu_j = \mathrm{E}\{X_j\}$
- $\boldsymbol{\Sigma}$ is the $p \times p$ covariance matrix. The (i,j)th element is $\sigma_{ij} = \mathrm{Cov}\{X_i, X_j\}$.

To get a better understanding of the MVN we focus on the exponential which has the factor

$$(\boldsymbol{x}-\boldsymbol{\mu})^t\boldsymbol{\Sigma}^{-1}(\boldsymbol{x}-\boldsymbol{\mu})$$

This factor

- is the only factor in the density function that depends on $\boldsymbol{x}$
- is a quadratic form
- is constant in points $\boldsymbol{x}$ with constant density $f(\boldsymbol{x})$.

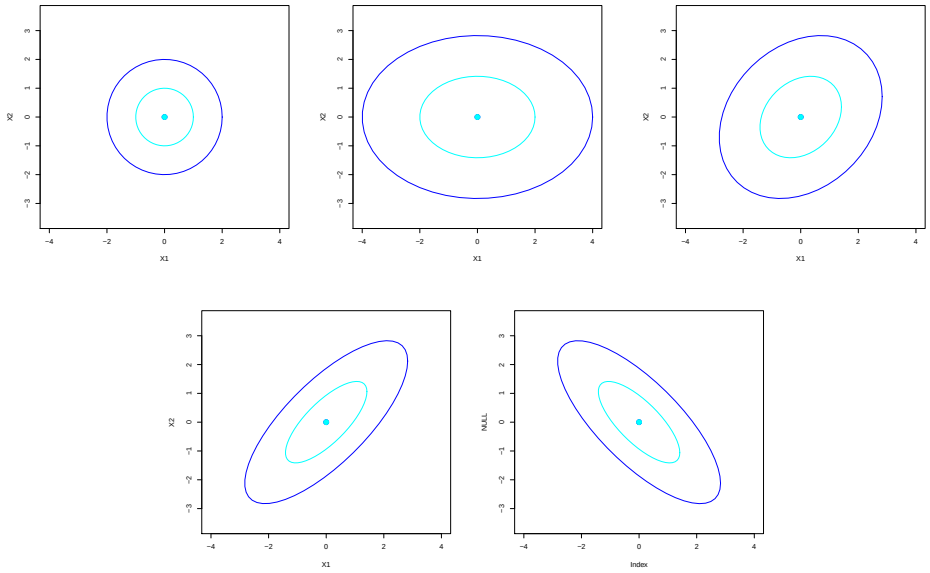
Consider $p = 2$ (bivariate normal). Then, all $\boldsymbol{x} \in \mathbb{R}^2$ for which

$$(\boldsymbol{x} - \boldsymbol{\mu})^t \boldsymbol{\Sigma}^{-1} (\boldsymbol{x} - \boldsymbol{\mu}) = \text{constant } c^2$$

lie on an ellipse with center $\boldsymbol{\mu}$.

These ellipses are known as constant density ellipses.

Some examples for different $\boldsymbol{\Sigma}$.



Now plug in the SVD of $\boldsymbol{\Sigma} = \boldsymbol{V}\boldsymbol{D}\boldsymbol{V}^t$,

$$\begin{aligned}(\boldsymbol{x}-\boldsymbol{\mu})^t\boldsymbol{\Sigma}^{-1}(\boldsymbol{x}-\boldsymbol{\mu}) &= c^2\\ (\boldsymbol{x}-\boldsymbol{\mu})^t\boldsymbol{V}\boldsymbol{D}^{-1}\boldsymbol{V}^t(\boldsymbol{x}-\boldsymbol{\mu}) &= c^2\end{aligned}$$

Note that $\boldsymbol{x}-\boldsymbol{\mu}$ is the centered $\boldsymbol{x}$. Without loss of generality, take $\boldsymbol{\mu} = \mathbf{0}$. Hence,

$$\begin{aligned}\boldsymbol{x}^t\boldsymbol{V}\boldsymbol{D}^{-1}\boldsymbol{V}^t\boldsymbol{x} &= c^2\\ \left(\boldsymbol{x}^t\boldsymbol{V}\right)\boldsymbol{D}^{-1}\left(\boldsymbol{x}^t\boldsymbol{V}\right)^t &= c^2\\ \sum_{j=1}^{p}(\boldsymbol{x}^t\boldsymbol{v}_j)^2/\delta_j &= c^2\\ \sum_{j=1}^{p}(z_j)^2/(c^2\delta_j) &= 1.\end{aligned}$$

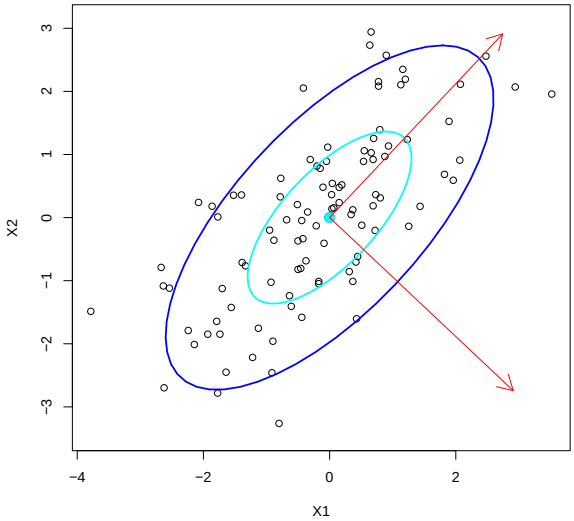
The last equation is the equation of an ellipse with axes parallel to the basis of $(z_1,\ldots,z_p)$ and with half axis lengths $c\sqrt{\delta_j} = c\lambda_j$ with λ_j the jth eigenvalue of $\boldsymbol{\Sigma}$.

We start from the last equation

$$\sum_{j=1}^{p}(z_j)^2/(c^2\delta_j) = 1,$$

which is the equation of an ellipse with axes parallel to the basis of $(z_1,\ldots,z_p)$ and with half axis lengths $c\sqrt{\delta_j} = c\lambda_j$ with λ_j the jth eigenvalue of $\boldsymbol{\Sigma}$.

Since $z_j = \boldsymbol{x}^t\boldsymbol{v}_j$, we conclude that the singular vectors $\boldsymbol{v}_j$ form the axes of the ellipse.



This graphs shows $n = 100$ data points on $p = 2$ dimensions. The data has been randomly sampled from a bivariate distribution. The two ellipses are constant density ellipses for two different values of c (i.e. two different constant densities). The inner ellipse corresponds to the largest constant density and the outer to the smallest constant density. The arrows show the two eigenvectors / singular vectors and they form the axes of the constant density ellipses.

In terms of data from a MVN a constant density ellipse may be interpreted as follows.

- take a point on the ellipse
- consider a small neighbourhood centered at this point (e.g. a small circle or square)
- count the number of data points in this neighborhood

If the data set is large, then it is expected that the density of data points is constant near the constant density ellipse. Thus if we repeat the procedure and move the neighbourhood along the ellipse, then it is expected that the number of points in those many neighbourhoods is more or less constant. The count of the number of points in the neighbourhood increases as the constant c increases (related to the constant density).

2.6.8 Biplots for PCA

Biplots for PCA

Because of the close connection between PCA and the SVD, the biplot as discussed before is still meaningful, with the first axis pointing into the direction of largest variance.

However, a modified version of the biplot may be considered to provide information on correlations/covariances between the original variables.

Consider the truncated SVD

$$\boldsymbol{X}_k = \boldsymbol{U}_k \boldsymbol{D}_k^{\alpha} \boldsymbol{D}_k^{1-\alpha} \boldsymbol{V}_k^t = \sum_{j=1}^{k} \left(\delta_j^{\alpha} \boldsymbol{u}_j\right) \left(\delta_j^{1-\alpha} \boldsymbol{v}_j\right)$$

where $0 \leq \alpha \leq 1$.

Note that $\boldsymbol{D}_k$ is a diagonal matrix with δ_j's on the diagonal. Hence, $\boldsymbol{D}_k^{\alpha}$ and $\boldsymbol{D}_k^{1-\alpha}$ are also diagonal matrices with elements δ_j^{α} and $\delta_j^{1-\alpha}$, respectively.

This suggests a scaled biplot. In particular,

$$\boldsymbol{X}_k = \boldsymbol{U}_k \boldsymbol{D}_k^{\alpha} \boldsymbol{D}_k^{1-\alpha} \boldsymbol{V}_k^t = \boldsymbol{Z}_k \left(\boldsymbol{D}_k^{1-\alpha} \boldsymbol{V}_k^t\right).$$

The scaled biplot in k dimensions consists of

- dots: the n rows of $\boldsymbol{Z}_k = \boldsymbol{U}_k \boldsymbol{D}_k^{\alpha}$
- vectors: the p rows of $\boldsymbol{V}_k \boldsymbol{D}_k^{1-\alpha}$

This biplot still posses the property that the orthogonal projection of a dot (of observation i) on a vector (of variable j) gives an approximation of the original observation i on variable j.

Note that the biplot we have seen before, corresponds to $\alpha = 1$.

Consider the case $\alpha = 0$. When the columns of $\boldsymbol{X}$ are centered, we find

$$(n-1)\boldsymbol{\Sigma}_X = \boldsymbol{X}^t\boldsymbol{X} = \boldsymbol{V}\boldsymbol{D}^2\boldsymbol{V}^t = (\boldsymbol{V}\boldsymbol{D})(\boldsymbol{V}\boldsymbol{D})^t$$

Thus, up to a factor $n-1$, element (i,j) of $\boldsymbol{\Sigma}_X$ (covariance between variable i and j) equals the orthogonal projection of the ith row of $\boldsymbol{V}\boldsymbol{D}$ onto the jth row of $\boldsymbol{V}\boldsymbol{D}$

The k-dimensional scaled biplot ($\alpha = 0$) shows the rows of $\boldsymbol{V}_k\boldsymbol{D}_k$ as vectors.

From PCA we know that $(n-1)\boldsymbol{\Sigma}_k = \boldsymbol{V}_k\boldsymbol{D}_k^2\boldsymbol{V}_k^t = (\boldsymbol{V}_k\boldsymbol{D}_k)(\boldsymbol{V}_k\boldsymbol{D}_k)^t$ is the best rank-k approximation of $(n-1)\boldsymbol{\Sigma}_X$ (in the sense that the Frobenius norm is mimimised).

Note that any choice of $\alpha \neq 1$ does no longer give the Euclidean distance interpretation between the rows of $\boldsymbol{Z}_k$.

2.7 Summary of the SVD

Some Final Comments on the SVD

Different ways of looking at the truncated SVD

$$\boldsymbol{X}_k = \boldsymbol{U}_k \boldsymbol{D}_k \boldsymbol{V}_k^t = \sum_{j=1}^{k} \delta_j \boldsymbol{u}_j \boldsymbol{v}_j.$$

Matrix Approximation Data compression from $n \times p$ entries in $\boldsymbol{X}$ to $k \times (1 + n + p)$ in $\boldsymbol{X}_k$ (actually less because of orthonormality constraints)

The matrix $\boldsymbol{X}_k$, which has rank $k < p$ is an example of a low-rank approximation of $\boldsymbol{X}$.

The matrix $\boldsymbol{X}_k$ is the best rank k approximation in terms of the Frobenius norm:

$$\text{Min}_{A:\text{rank}(A)=k} \|\boldsymbol{X} - \boldsymbol{A}\|_F^2 = \|\boldsymbol{X} - \boldsymbol{X}_k\|_F^2 = \sum_{j=k+1}^{r} \delta_j^2.$$

Some Final Comments on the SVD

Different ways of looking at the truncated SVD

$$\boldsymbol{X}_k = \boldsymbol{U}_k \boldsymbol{D}_k \boldsymbol{V}_k^t = \sum_{j=1}^{k} \delta_j \boldsymbol{u}_j \boldsymbol{v}_j.$$

Distance Approximation The n rows of $\boldsymbol{X}_k$ lie in a k-dimensional subspace of $\mathbb{R}^p$, i.e. a Dimension reduction from p to $k \leq \text{rank}(\boldsymbol{X})$.

The distances between the n observations in $\boldsymbol{X}$ are best approximated in a k-dimensional subspace by $\boldsymbol{X}_k$, in the following sense

$$\boldsymbol{X}_k = \text{ArgMin}_{A:\text{rank}(A)=k} \|\boldsymbol{D}(\boldsymbol{X}) - \boldsymbol{D}(\boldsymbol{A})\|_F^2,$$

or, equivalently,

$$\boldsymbol{X}_k = \text{ArgMin}_{A:\text{rank}(A)=k} \|\boldsymbol{X}\boldsymbol{X}^t - \boldsymbol{A}\boldsymbol{A}^t\|_F^2.$$

Different ways of looking at the truncated SVD

$$\boldsymbol{X}_k = \boldsymbol{U}_k \boldsymbol{D}_k \boldsymbol{V}_k^t = \sum_{j=1}^{k} \delta_j \boldsymbol{u}_j \boldsymbol{v}_j.$$

Covariance Matrix Approximation The n rows of $\boldsymbol{X}_k$ lie in a k-dimensional subspace of $\mathbb{R}^p$, i.e. a Dimension reduction from p to $k \leq \text{rank}(\boldsymbol{X})$.

The covariances between the n observations in $\boldsymbol{X}$ are best approximated in a k-dimensional subspace by $\boldsymbol{X}_k$, in the following sense

$$\boldsymbol{X}_k = \text{ArgMin}_{A:\text{rank}(A)=k} \| \text{Var}\{\boldsymbol{X}\} - \text{Var}\{\boldsymbol{A}\} \|_F^2,$$

or, equivalently,

$$\boldsymbol{X}_k = \text{ArgMin}_{A:\text{rank}(A)=k} \| \boldsymbol{X}^t\boldsymbol{X} - \boldsymbol{A}^t\boldsymbol{A} \|_F^2.$$

Different ways of looking at the truncated SVD

$$\boldsymbol{X}_k = \boldsymbol{U}_k \boldsymbol{D}_k \boldsymbol{V}_k^t = \sum_{j=1}^{k} \delta_j \boldsymbol{u}_j \boldsymbol{v}_j.$$

Dimension Reduction Transformation with Maximal Variance

The identities

$$\boldsymbol{X}_k = \boldsymbol{Z}_k \boldsymbol{V}_k^t \text{ and } \boldsymbol{Z}_k = \boldsymbol{X}\boldsymbol{V}_k$$

show that the original n p-dim. observations in $\boldsymbol{X}$ can be transformed to k-dim. observations in $\boldsymbol{Z}_k$.

The transformation $\boldsymbol{V}$ is such that

- $\boldsymbol{V}^t\boldsymbol{V} = \boldsymbol{I}$ (orthonormal)
- $\text{Var}\{\boldsymbol{Z}_k\}$ is a diagonal matrix (uncorrelated PCs)
- variances of the k PCs are maximal.

The terminology "scores" and "loadings" is very conventional in PCA.

2.8 A Sparse Singular Value Decomposition

2.8.1 Introduction

Sparse SVD

In high dimensional data the PCs may have unclear interpretation because they depend on all p original variables.

Example: toxicogenomics in early drug development

- 30 chemical compounds have been screened for toxicity
- several bioassays have been developed for toxicity screening
- Gene expressions in a liver cell line are measured for each compound (4000 genes)

The toxicogenomics data set thus contains $n = 30$ observations (compounds) on $p = 4000$ features (genes). The toxicity score is actually measured on a continuous scale, but here we will make it a binary score (toxic and non-toxic).

We now apply a SVD on the data matrix $\boldsymbol{X}$ and look at the squared singular values (= eigenvalues of the covariance matrix). Note that the rank of $\boldsymbol{X} \leq \min(30, 4000) = 30$.

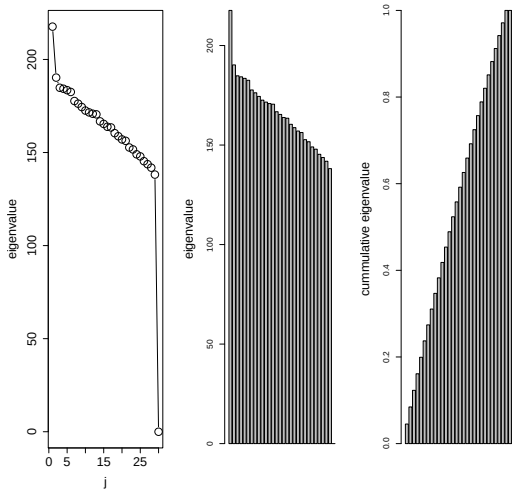
Example: Toxicogenomics

```
load("../Data/X.RData") # centered gene expression matrix
load("../Data/Y.RData") # toxicity
rownames(X)<-paste("C",1:30,sep="")
tox<-ifelse(Y<0,1,2)

X.svd<-svd(X)

plot(X.svd$d^2,type="b",ylab="eigenvalue",xlab="j",cex=2,
     cex.axis=1.5,cex.lab=1.5)
barplot(X.svd$d^2,ylab="eigenvalue",cex.lab=1.5)
barplot(cumsum(X.svd$d^2)/sum(X.svd$d^2),
     ylab="cummulative eigenvalue",cex.lab=1.5)
```

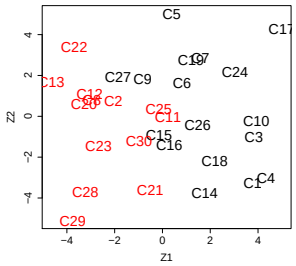
Example: Toxicogenomics



From these graphs we conclude that all 30 dimensions are important. Which perhaps comes as no surprise, given that we start from 4000 features.

Example: Toxicogenomics

```
k<-2
Uk<-X.svd$u[,1:k]
Dk<-diag(X.svd$d[1:k])
Zk<-Uk%*%Dk
Vk<-X.svd$v[,1:k]
plot(Zk,xlab="Z1",ylab="Z2",type="n")
text(Zk,rownames(X),col=tox)
```

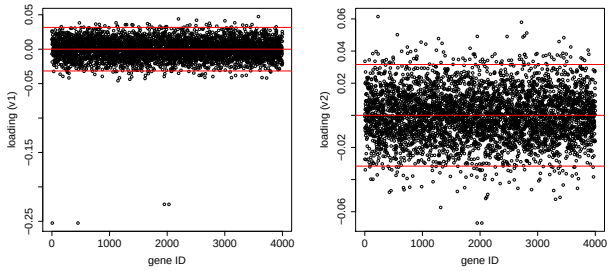


The graph shows the scores on the first two principal components (or MDS plot). Each point corresponds to a compound. The color code refers to the toxicity score (black=non-toxic; red=toxic). The plot shows a clear separation between toxic and non-toxic compounds.

The next logical step in a PCA is to interpret the principal components (i.e. the two axis of the plot). We thus need to look at the loadings. We can add a vector for each gene to the graph so as to get a biplot, but this would require plotting 4000 vectors. This would of course make the biplot unreadable. The next slide shows an alternative graph to look at the many loadings of the first two PCs.

Example: Toxicogenomics

```
plot(Vk[,1],cex=0.8,cex.axis=1.5,cex.lab=1.5,xlab="gene ID",
     ylab="loading (v1)")
abline(h=0,col=2,lwd=2)
abline(h=2*sd(Vk[,1]),col=2,lwd=2)
abline(h=-2*sd(Vk[,1]),col=2,lwd=2)
```



$\longrightarrow$ PCs hard to interpret because all 4000 genes are involved

From these graphs we see, for each PC, 4000 non-zero loadings. It is almost impossible to get a good interpretation: which genes are important and lead the interpretation of the PCs?

In an attempt to find the most important genes (in the sense that they drive the interpretation of the PCs), the plots show horizontal reference lines: the average of the loadings, and the average $\pm$ twice the standard deviation of the loadings. In between the lines we expects about 95% of the loadings (if they were normally distributed). The points outside the band come from the genes that have rather large loadings (in absolute value) and hence are important for the interpretation of the PCs.

Note that particularly for the first PC, only a few genes show a markedly large loading. These genes will very likely play an important role in the toxicity mechanism (because difference between toxic / non-toxic compounds can be particularly seen from the first PC). Because these genes have a large negative loading, the genes are up-regulated (i.e. show large expression) for the toxic compounds (which show a negative score on the first PC).

Basic idea behind a sparse matrix decomposition of an $n \times p$ matrix $\boldsymbol{X}$

Find a rank k approximation of $\boldsymbol{X}$ of the form

$$\boldsymbol{X}_k = \sum_{j=1}^{k} \delta_j \boldsymbol{u}_j \boldsymbol{v}_j^t$$

such that many of the elements in the $\boldsymbol{v}_j$ and or $\boldsymbol{u}_j$ are exactly zero.

Here we focus on the Penalised Matrix Decomposition (PMD) method of Witten *et al.* (2009, Biostatistics, **10**, 3, 515-534).

When many of the loadings in $\boldsymbol{v}_j$ are zero, the interpretation of the PC only depends on the features that correspond to the remaining non-zero loadings.

2.8.2 Penalised Matrix Decomposition

Penalised Matrix Decomposition

The rank-1 PMD approximation of the $n \times p$ matrix $\boldsymbol{X}$ is of the form

$$\boldsymbol{X}_1 = \delta_1 \boldsymbol{u}_1 \boldsymbol{v}_1^t,$$

and it is the solution to the following minimisation problem:

$$\min_{\delta,u,v} \|\boldsymbol{X} - \delta \boldsymbol{u}\boldsymbol{v}^t\|_F^2 \text{ subject to } \delta \geq 0, \|\boldsymbol{u}\|_2^2 = \|\boldsymbol{v}\|_2^2 = 1, |\boldsymbol{u}| \leq c_1, |\boldsymbol{v}| \leq c_2$$

for some user-defined constants $c_1, c_2 > 0$.

The constraints $|\boldsymbol{u}| \leq c_1$ and $|\boldsymbol{v}| \leq c_2$ are known as L_1 or lasso constraints.

Note,

$$|\boldsymbol{u}| = \sum_{i=1}^{n} |u_i| \text{ and } |\boldsymbol{v}| = \sum_{i=1}^{p} |v_i|.$$

If $c_1 = +\infty$, then the constraint on $\boldsymbol{u}$ is removed (similar for c_2).

Note that

$$\min_{\delta,u,v} \|\boldsymbol{X} - \delta \boldsymbol{u}\boldsymbol{v}^t\|_F^2 = \min_{A:\text{rank}(A)=1} \|\boldsymbol{X} - \boldsymbol{A}\|_F^2$$

with $\boldsymbol{A} = \boldsymbol{X}_1$ (truncated SVD of $\boldsymbol{X}$, truncated after 1 term). However, the PMD requires a constrained minimisation. The unconstrained solution arises with $c_1 = c_2 = +\infty$.

Note that $\|\boldsymbol{u}\|_2^2 = \|\boldsymbol{v}\|_2^2 = 1$ may also be written as $\boldsymbol{u}^t\boldsymbol{u} = \boldsymbol{v}^t\boldsymbol{v} = 1$ (i.e. the vectors are normalised, just like for the singular vectors of the SVD).

We have also applied the L_1 constraint to the least-squares estimator of the regression parameters in a linear regression model. The effect was that many of the parameter estimates are set exactly to zero. Here, the same effect will take place. Many of the elements in $\boldsymbol{u}$ and $\boldsymbol{v}$ will be set exactly to zero. In particular, the smaller c_2, the more elements of $\boldsymbol{v}$ will be set to zero (shrinkage). Similarly, the smaller c_1, the more elements of $\boldsymbol{u}$ will be set to zero. The choice of c_1 and c_2 depends on the objective of the data analysis and will be discussed later.

The solution is known as a rank-1 PMD approximation of $\boldsymbol{X}$ because $\delta_1 \boldsymbol{u}_1 \boldsymbol{v}_1^t$ is a matrix of rank 1.

The rank-2 PMD approximation is found as follows:

1. find the rank-1 PMD as before: $\boldsymbol{X}_1 = \delta_1 \boldsymbol{u}_1 \boldsymbol{v}_1^t$;
2. compute
$$\tilde{\boldsymbol{X}}_2 = \boldsymbol{X} - \boldsymbol{X}_1,$$
and compute the rank-1 PMD of $\tilde{\boldsymbol{X}}_2$, which gives $\delta_2, \boldsymbol{u}_2, \boldsymbol{v}_2$;
3. the rank-2 PMD of $\boldsymbol{X}$ is then given by
$$\boldsymbol{X}_2 = \delta_1 \boldsymbol{u}_1 \boldsymbol{v}_1^t + \delta_2 \boldsymbol{u}_2 \boldsymbol{v}_2^t.$$

Note that the $\boldsymbol{u}$s and $\boldsymbol{v}$s are not necessarily orthogonal (only if $c_1, c_2 = +\infty$).

The rank-k PMD approximation is found as follows:

1. find the rank-1 PMD as before: $\tilde{\boldsymbol{X}}_1 = \delta_1 \boldsymbol{u}_1 \boldsymbol{v}_1^t$;
2. set $j = 1$;
3. compute
$$\tilde{\boldsymbol{X}}_{j+1} = \boldsymbol{X} - \tilde{\boldsymbol{X}}_1 - \cdots - \tilde{\boldsymbol{X}}_j,$$
and compute the rank-1 PMD of $\tilde{\boldsymbol{X}}_{j+1}$, which gives $\delta_{j+1}, \boldsymbol{u}_{j+1}, \boldsymbol{v}_{j+1}$;
4. if $j < k$, increase j with one and go back to step 3; otherwise go to step 5;
5. the rank-k PMD of $\boldsymbol{X}$ is then given by
$$\boldsymbol{X}_k = \sum_{j=1}^{k} \delta_j \boldsymbol{u}_j \boldsymbol{v}_j^t.$$

Without the L_1 constraints, this algorithm gives the ordinary SVD. As before the δ_js, $\boldsymbol{u}_j$s and $\boldsymbol{v}_j$s are stacked into matrices $\boldsymbol{D}_k$, $\boldsymbol{U}_k$ and $\boldsymbol{V}_k$: $\boldsymbol{X}_k = \boldsymbol{U}_k \boldsymbol{D}_k \boldsymbol{V}_k^t$.

Note that the $\boldsymbol{u}$ vectors are generally not orthogonal (i.e. $\boldsymbol{u}_i^t \boldsymbol{u}_j \neq 0$ for $i \neq j$) (the same holds for the $\boldsymbol{v}$ vectors).

2.8.3 Sparse PCA via PMD

Sparse PCA via PMD

Just like the SVD can be used to find the PCA solution, the PMD can be used to find a sparse PCA solution (SPC).

For a PCA, it is only important to have sparse loadings (in the $\boldsymbol{v}$s); the scores (involving the $\boldsymbol{u}$s) need not be sparse.

The rank-1 solutions thus satisfy

$$\min_{\delta,u,v} \|\boldsymbol{X} - \delta \boldsymbol{u}\boldsymbol{v}^t\|_F^2 \text{ subject to } \delta \geq 0, \|\boldsymbol{u}\|_2^2 = \|\boldsymbol{v}\|_2^2 = 1, |\boldsymbol{v}| \leq c$$

(i.e. no sparsity constraint on $\boldsymbol{u}$).

Hence the PCA scores

$$\boldsymbol{Z}_k = \boldsymbol{U}_k \boldsymbol{D}_k$$

relate to $\boldsymbol{X}_k$ through the sparse loadings in $\boldsymbol{V}_k$.

It can be shown that the SPC solution (rank-1) also maximises

$$\boldsymbol{v}^t \boldsymbol{X}^t \boldsymbol{X} \boldsymbol{v} \text{ subject to } \|\boldsymbol{u}\|_2^2 = \|\boldsymbol{v}\|_2^2 = 1, |\boldsymbol{v}| \leq c$$

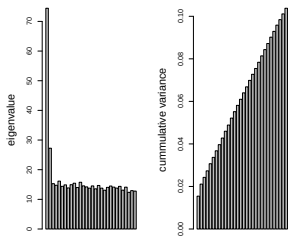
(i.e. it maximises the variance of the PCs, subject to sparsity constraint on loadings).

From these two slides we may conclude that we can use the PMD as the basis of a sparse PCA, and that the interpretation is as before in terms of maximising variance. Only now we hope that only a few features will give a non-zero loading in the $\boldsymbol{v}$-vectors. Biplots may again be constructed (with fewer vectors to be plotted).

We re-analyse the toxicogenomics data with a sparse PCA. We set $c_2 = 5$ (arbitrary choice).

Example: Toxicogenomics

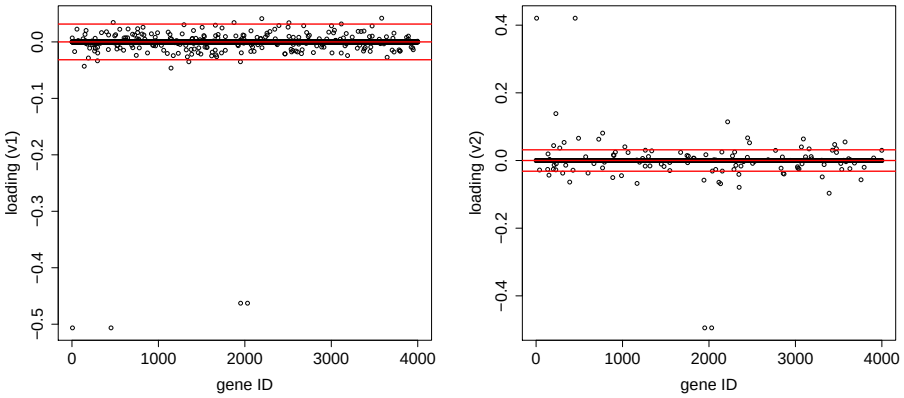
```
library(PMA)
X.spc<-SPC(X,K=30,sumabsv = 5)
par(mfrow=c(1,2))
barplot(X.spc$d^2,ylab="eigenvalue",cex.lab=1.5)
barplot(X.spc$prop.var.explained,
     ylab="cummulative variance",cex.lab=1.5)
```



The plot on the left shows the eigenvalues (squared singular values) from the PMD. This suggests that the first one or two are the most important, but all eigenvalues remain substantially larger than zero. The figure on the right shows the cumulative percentage of variance retained in the PCs. These percentages are now no longer calculated as before as ratio's of sums of eigenvalues. A more complicated calculation is required now, because the PCs are no longer uncorrelated with one another. Thus part of the information (variance) of one PC is shared with part of the information of another PCs (i.e. non-zero correlation or covariance).

Example: Toxicogenomics

Loadings of the first two PCs, with constraint $|\boldsymbol{v}| < 5$.

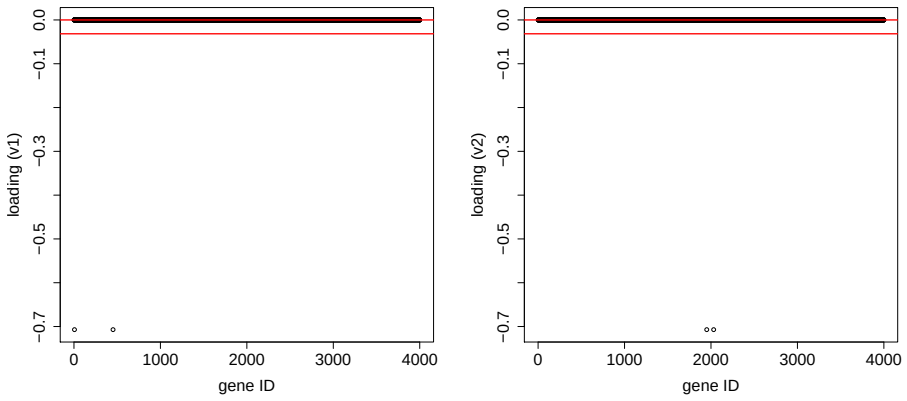


These plots show the loadings of the first two sparse PCs. Many of the loadings are now exactly equal to zero. Only few genes show large loadings.

We now repeat the analysis, but with the constraint $|\boldsymbol{v}| < 1$. This gives a much sparser solution with only two genes with non-zero loadings in each dimension.

Example: Toxicogenomics

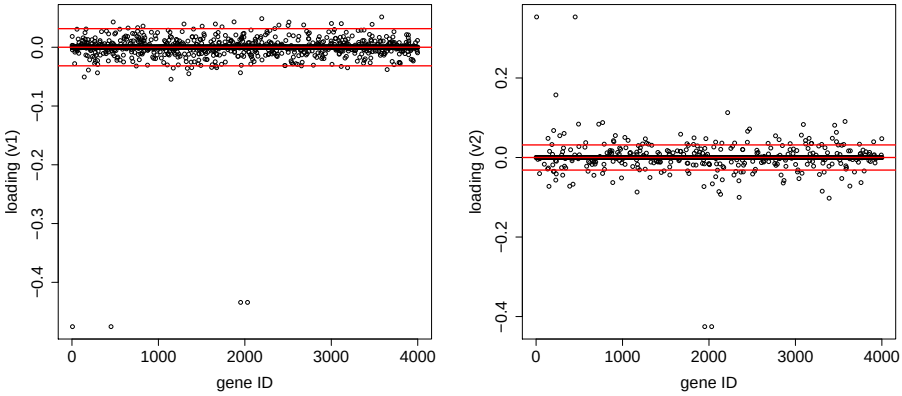
Loadings of the first two PCs, with constraint $|\boldsymbol{v}| < 1$.



We now repeat the analysis, but with the constraint $|\boldsymbol{v}| < 10$. This gives a much less sparse solution with several genes with non-zero loadings.

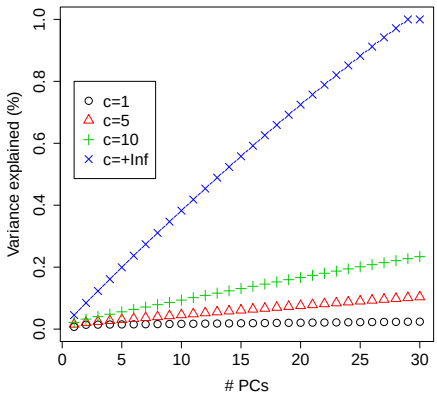
Example: Toxicogenomics

Loadings of the first two PCs, with constraint $|\boldsymbol{v}| < 10$.



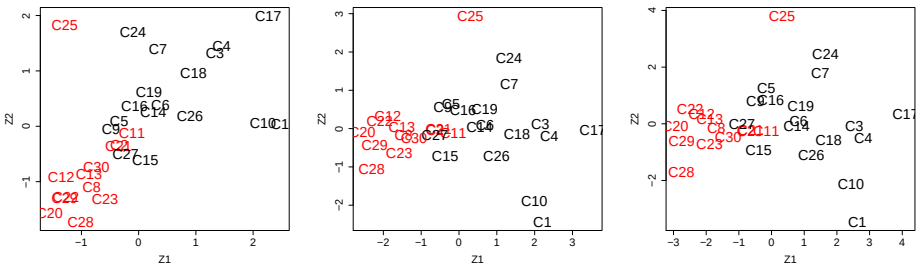
Example: Toxicogenomics

Variance explained for $c = 1, 5, 10$ and $+\infty$.



This graph illustrates that the percentages of variance explained by the PCs are smaller for sparser solutions. Thus in choosing an appropriate penalty parameter c_2 one should not only look at the sparseness of the solution, but also at the percentage of variance contained in the first few PCs. If this percentage is very small, then the graphs of the scores (or biplots) are no longer very informative (i.e. they will not tell the full story). Thus a compromise must be looked for.

Example: Toxicogenomics



Finally we show the plots of the scores on the first two sparse PCs for $c_2 = 1$ (left), $c_2 = 5$ (middle) and $c_2 = 10$ (right).

Note that the most sparse solution (left) shows a strong positive correlation between the first two PCs (as a consequence of the loss of orthogonality of the singular vectors).

All three graphs still succeed quite well in separating the toxic from the non-toxic genes. Hence we may expect that the two genes identified from the most sparse solution play an important role in the development of toxic effects. Note, however, that our data analysis did not provide a scientific proof for this conclusion: (sparse) PCA is only an exploratory or descriptive data analysis method that helps the data-analyst to gain insight into the data. The fact that we found a few genes that appear to be associated with toxicity is of course a nice result. It is important to understand that the toxicity outcome was not used in the (sparse) PCA. We only used the toxicity to color the points in the graphs. PCA is an unsupervised method. Later we will discuss supervised methods (and these methods will use the toxicity outcome data so that outcome-associated genes can be better selected).

3 Linear Discriminant Analysis

Linear Discriminant Analysis (LDA)



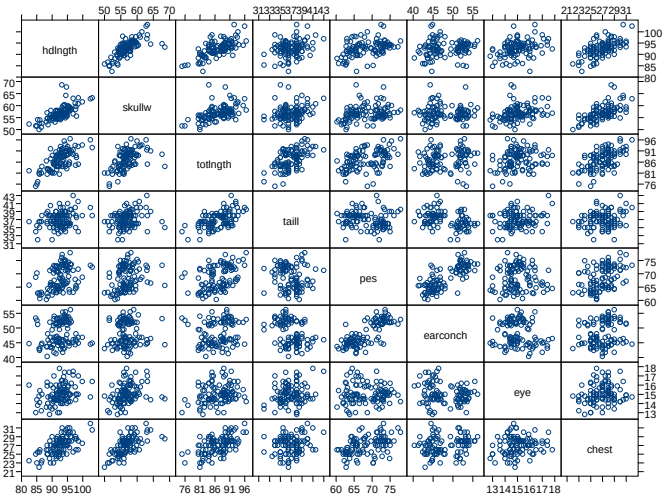
possum − small marsupial species native to Australia

The data set consists of morphological measurements on 103 possums that were captured in 7 south eastern regions in Australia. The 8 morphological variables are:

- head length (hdlngth)
- skull width (skullw)
- total length (totlngth)
- tail length (taill)
- feet length (pes)
- ear length (earconch)
- eye size (eye)
- chest girth (chest)



The scatterplot matrix given on the next slide is only gives for data exploration purposes only.



The objectives of this study are summarised as:

- how do the 7 geographical groups differ from one another in terms of the morphology?

This objective is also known as *discrimination*, which is used here as a targeted data exploration analysis. Discrimination only aims at understanding how the k groups may differ from one another in terms of the X-variables. In classification, on the other hand, it is of less importance to understand how the predictors contribute to good classifications; it is bigger importance that the prediction models performs well in predicting/classifying new subjects to the correct class.

Fisher's construction of LDA is simple: it allows for classification in a dimension-reduced subspace of $\mathbb{R}^p$.

First we look at the setting with only two groups.

Fisher looked for a direction, say $\boldsymbol{a}$, in the p-dimensional predictor space such that the orthogonal projections of the predictors, $z = \boldsymbol{x}^t\boldsymbol{a}$, show maximal ratio between the between and within sums of squares:

$$\boldsymbol{v} = \text{ArgMax}_a \frac{\text{SSB}_z}{\text{SSE}_z}$$

Some more details on the ratio between the between and the within sums of squares.

- let $\boldsymbol{x}_{ik}$ denote the ith p-dimensional observation in the kth group ($k = 0, 1$), $i = 1, \ldots, n_k$.
- let $z_{ik} = \boldsymbol{a}^t\boldsymbol{x}_{ik}$ denote the orthogonal projection of $\boldsymbol{x}_{ik}$ onto $\boldsymbol{a}$
- for the one-dimensional z-observations, consider the following sum of squares:

$$\text{SSE}_z = \text{within sum of squares} = \sum_{k=0,1}\sum_{i=1}^{n_k}(z_{ik} - \bar{z}_k)^2$$

$$\text{SSB}_z = \text{between sum of squares} = \sum_{k=0,1}\sum_{i=1}^{n_k}(\bar{z}_k - \bar{z})^2 = \sum_{k=0,1} n_k(\bar{z}_k - \bar{z})^2$$

with $\bar{z}_k$ the sample mean of z_{ik} within group k, and $\bar{z}$ the sample mean of all z_{ik}.

- to reformulate SSE_z and SSB_z in terms of the p-dimensional $\boldsymbol{x}_{ik}$, we need the sample means

$$\bar{z}_k = \frac{1}{n_k}\sum_{i=1}^{n_k} z_{ik} = \frac{1}{n_k}\sum_{i=1}^{n_k} \boldsymbol{a}^t\boldsymbol{x}_{ik} = \boldsymbol{a}^t\frac{1}{n_k}\sum_{i=1}^{n_k}\boldsymbol{x}_{ik} = \boldsymbol{a}^t\bar{\boldsymbol{x}}_k$$

$$\bar{z} = \frac{1}{n}\sum_{k=0,1}\sum_{i=1}^{n_k} z_{ik} = \cdots = \boldsymbol{a}^t\bar{\boldsymbol{x}}.$$

- SSE_z becomes

$$\text{SSE}_z = \sum_{k=0,1}\sum_{i=1}^{n_k}(z_{ik}-\bar{z}_k)^2 = \boldsymbol{a}^t\left(\sum_{k=0,1}\sum_{i=1}^{n_k}(\boldsymbol{x}_{ik}-\bar{\boldsymbol{x}}_k)(\boldsymbol{x}_{ik}-\bar{\boldsymbol{x}}_k)^t\right)\boldsymbol{a}$$

- SSB_z becomes

$$\text{SSB}_z = \sum_{k=0,1} n_k(\bar{z}_k-\bar{z})^2 = \boldsymbol{a}^t\left(\sum_{k=0,1} n_k(\bar{\boldsymbol{x}}_k-\bar{\boldsymbol{x}})(\bar{\boldsymbol{x}}_k-\bar{\boldsymbol{x}})^t\right)\boldsymbol{a}$$

- The $p\times p$ matrix

$$\boldsymbol{W} = \sum_{k=0,1}\sum_{i=1}^{n_k}(\boldsymbol{x}_{ik}-\bar{\boldsymbol{x}}_k)(\boldsymbol{x}_{ik}-\bar{\boldsymbol{x}}_k)^t$$

  is referred to as the matrix of within sum of squares and cross products.

- The $p\times p$ matrix

$$\boldsymbol{B} = \sum_{k=0,1} n_k(\bar{\boldsymbol{x}}_k-\bar{\boldsymbol{x}})(\bar{\boldsymbol{x}}_k-\bar{\boldsymbol{x}})^t$$

  is referred to as the matrix of between sum of squares and cross products.

Note that on the diagonal of $\boldsymbol{W}$ and $\boldsymbol{B}$ you find the ordinary univariate within and between sums of squares of the individual components of $\boldsymbol{x}$.

With the new notation, the optimisation problem may be formulated as:

$$\boldsymbol{v} = \text{ArgMax}_a \frac{\boldsymbol{a}^t \boldsymbol{B} \boldsymbol{a}}{\boldsymbol{a}^t \boldsymbol{W} \boldsymbol{a}}.$$

To guarantee a unique meaningful solution a restriction on $\boldsymbol{a}$ is needed:

$$\boldsymbol{a}^t \boldsymbol{W} \boldsymbol{a} = 1.$$

The optimisation problem thus becomes:

$$\boldsymbol{v} = \text{ArgMax}_a \frac{\boldsymbol{a}^t \boldsymbol{B} \boldsymbol{a}}{\boldsymbol{a}^t \boldsymbol{W} \boldsymbol{a}} \text{ subject to } \boldsymbol{a}^t \boldsymbol{W} \boldsymbol{a} = 1.$$

An equivalent formulation:

$$\boldsymbol{v} = \text{ArgMax}_a \boldsymbol{a}^t \boldsymbol{B} \boldsymbol{a} \text{ subject to } \boldsymbol{a}^t \boldsymbol{W} \boldsymbol{a} = 1.$$

This can be solved by introducing a Langrange multiplier:

$$\boldsymbol{v} = \text{ArgMax}_a \boldsymbol{a}^t \boldsymbol{B} \boldsymbol{a} - \lambda(\boldsymbol{a}^t \boldsymbol{W} \boldsymbol{a} - 1).$$

Calculating the partial derivative w.r.t. $\boldsymbol{a}$ and setting it to zero gives

$$\begin{aligned} 2\boldsymbol{B}\boldsymbol{a} - 2\lambda \boldsymbol{W}\boldsymbol{a} &= 0 \\ \boldsymbol{B}\boldsymbol{a} &= \lambda \boldsymbol{W}\boldsymbol{a} \\ \boldsymbol{W}^{-1}\boldsymbol{B}\boldsymbol{a} &= \lambda \boldsymbol{a}. \end{aligned}$$

From the final equation we recognise that $\boldsymbol{v} = \boldsymbol{a}$ is an eigenvector of $\boldsymbol{W}^{-1}\boldsymbol{B}$, and λ is the corresponding eigenvalue. The equation has in general rank$(\boldsymbol{W}^{-1}\boldsymbol{B})$ solutions. In the case of two classes, the rank equals 1 and thus only one solution exists.

With more than two classes, say m classes, Fisher's method is constructed in exactly the same way. Now

$$\boldsymbol{W}^{-1}\boldsymbol{B}\boldsymbol{a} = \lambda\boldsymbol{a}$$

will have $r = \text{rank}(\boldsymbol{W}^{-1}\boldsymbol{B}) = \min(m-1, p, n)$ solutions (eigenvectors and eigenvalues). (n: sample size of training data)

Let $\boldsymbol{v}_j$ and λ_j denote the r solutions, and define

- $\boldsymbol{V}$: $p \times r$ matrix with collums $\boldsymbol{v}$
- $\boldsymbol{L}$: $r \times r$ diagonal matrix with elements $\lambda_1 > \lambda_2 > \cdots > \lambda_r$

The p-dimensional predictor data in $\boldsymbol{X}$ may then be transformed to the r-dimensional scores

$$\boldsymbol{Z} = \boldsymbol{X}\boldsymbol{V}.$$

Just as with PCA, the columns of $\boldsymbol{V}$ can be interpreted (cfr. loadings in PCA).

For eigenvectors $\boldsymbol{v}_i$ and $\boldsymbol{v}_j$, it holds that

$$\text{Cov}\{Z_i, Z_j\} = \text{Cov}\{\boldsymbol{X}\boldsymbol{v}_i, \boldsymbol{X}\boldsymbol{v}_j\} = \boldsymbol{v}_i^t \boldsymbol{W} \boldsymbol{v}_j = \delta_{ij},$$

in which the covariances are defined within groups. Hence, within the groups (classes) the scores are uncorrelated.

The next slide shows how the eigenvectors can be interpreted.

The r equations

$$\boldsymbol{W}^{-1}\boldsymbol{B}\boldsymbol{v} = \lambda \boldsymbol{v}$$

can be written as (upon using the $\boldsymbol{V}$ matrix)

$$\boldsymbol{W}^{-1}\boldsymbol{B}\boldsymbol{V} = \boldsymbol{V}\boldsymbol{L}.$$

Multiply LHS and RHS with $\boldsymbol{V}^t\boldsymbol{W}$ gives

$$\boldsymbol{V}^t\boldsymbol{B}\boldsymbol{V} = \boldsymbol{V}^t\boldsymbol{W}\boldsymbol{V}\boldsymbol{L} = \boldsymbol{L}$$

(we made use of the restriction $\boldsymbol{V}^t\boldsymbol{W}\boldsymbol{V} = \boldsymbol{I}$).

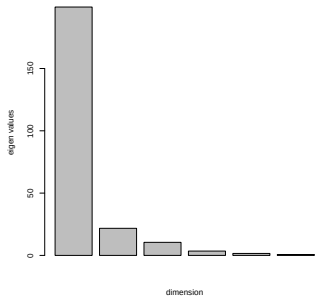
Hence, λ_j equals the maximised ratio

$$\frac{\boldsymbol{v}_j^t \boldsymbol{B} \boldsymbol{v}_j}{\boldsymbol{v}_j^t \boldsymbol{W} \boldsymbol{v}_j}.$$

With this interpretation of the eigenvalues, we can compare eigenvalues. For example, if the first eigenvalue is twice as large as the second, then we could say that the first dimension (Z_1) has a between over within sum of squares ratio that is twice as large as this ratio for the second dimension (Z_2).

Example: possums

```
> table(possum$site)
 1  2  3  4  5  6  7
33 12  7  7 13 13 18
> possum.fda<-lda(site~hdlngth+skullw+totlngth+taill+pes+earconch+
+                   eye+chest+belly,data=possum,prior=rep(1,7)/7)
> barplot(possum.fda$svd^2,ylab="eigenvalue",
+         xlab="dimension")
```



Fisher's LDA is illustrated on the possum data.

Note that the lda function gives singular values instead of eigenvalues. The latter is the square of the former.

The barplot of the eigenvalues indicate that the first dimension is obviously the most important dimension for discriminating between the 7 regions.

The eigenvalue λ_j can be interpreted as the ratio

$$\frac{\boldsymbol{v}_j^t \boldsymbol{B} \boldsymbol{v}_j}{\boldsymbol{v}_j^t \boldsymbol{W} \boldsymbol{v}_j},$$

or (upon using $\boldsymbol{v}_j^t \boldsymbol{W} \boldsymbol{v}_j = 1$) the between-centroid sum of squares (in the reduced dimension space of the Fisher discriminants)

$$\boldsymbol{v}_j^t \boldsymbol{B} \boldsymbol{v}_j.$$

```
> possum.fda
Call:
lda(site ~ hdlngth + skullw + totlngth + taill + pes + earconch +
    eye + chest + belly, data = possum, prior = rep(1, 7)/7)

Prior probabilities of groups:
        1         2         3         4         5         6
7
0.1428571 0.1428571 0.1428571 0.1428571 0.1428571 0.1428571 0.1428571

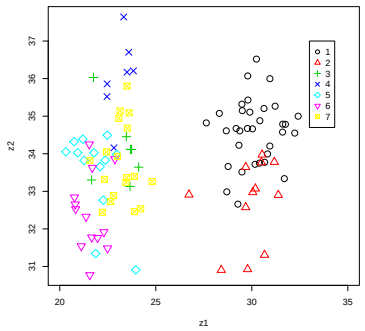
...
```

```
Coefficients of linear discriminants:
                 LD1         LD2         LD3         LD4
hdlngth   -0.14559112  0.12897836 -0.21020780  0.04898217
skullw    -0.02129290  0.13280218 -0.18278085  0.19730407
totlngth   0.06491626  0.12374944  0.38191525 -0.19287668
taill     -0.42281160  0.07756109 -0.53954957 -0.18794291
pes        0.30671358  0.01478734 -0.02893259  0.02832215
earconch   0.58825765  0.03134260 -0.04837783 -0.05052205
eye       -0.09187877 -0.16914125  0.76518802  0.20932923
chest      0.08187448  0.13160193  0.03271641 -0.13471556
belly      0.01037390 -0.08621817  0.12012950  0.25150135

Proportion of trace:
   LD1    LD2    LD3    LD4    LD5    LD6
0.8399 0.0916 0.0442 0.0146 0.0066 0.0030
```

The last line gives the relative importance of the dimensions. For example, $\lambda_1^2 / \sum_j \lambda_j^2 = 0.8399 \approx 84\%$.

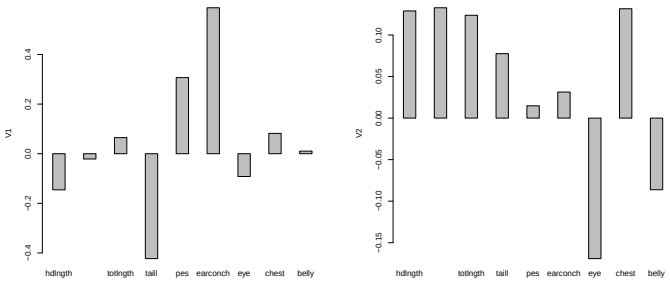
```
> V<-possum.fda$scaling[,1:2]
> Z<-as.matrix(possum[,6:14])%*%V
> plot(Z[,1],Z[,2],col=possum$site,pch=possum$site,
+      xlim=c(20,35),xlab="z1",ylab="z2",cex=1.5)
> legend(33,37,legend=1:7,pch=1:7,col=1:7)
```



The columns of the matrix $\boldsymbol{V}$ contain the eigenvectors. There are $\min(7-1, 8, 103) = 6$ eigenvectors. The 103×2 matrix $\boldsymbol{Z}$ contains the scores on the two Fisher discriminants.

The scores plotted in the first two dimensions show a nice separation between regions 1 and 2 on the one hand, and the other 5 regions on the other hand. These 5 other regions cannot be separated well.

```
> barplot(V[,1],ylab="V1",space=1.5)
> barplot(V[,2],ylab="V2",space=1.5)
```



These graphs visualise the elements in the first two columns of $\boldsymbol{V}$. These columns specify the linear combinations of the columns of $\boldsymbol{X}$. For example, $\boldsymbol{v}_1$ is the first column of $\boldsymbol{V}$. This vector is used for transforming the rows of $\boldsymbol{X}$: $\boldsymbol{z}_{i1} = \boldsymbol{x}_i^t \boldsymbol{v}_1$.

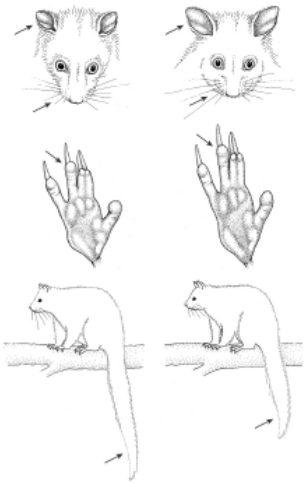
The plot for $\boldsymbol{v}_1$, tells us that the separation of regions 1 and 2 from the other groups (see previous slide), can be particularly explained by contrast.

Fisher's discriminant functions can be interpreted.

The first function read (approximately) as

$$Z_1 = -0.42 \times \mathsf{taill} + 0.31 \times \mathsf{pes} + 0.59 \times \mathsf{earconch}$$

$\longrightarrow$ possums from regions 1 and 2 have a shorter tail and bigger ears and feet than the possums from the other 5 regions.



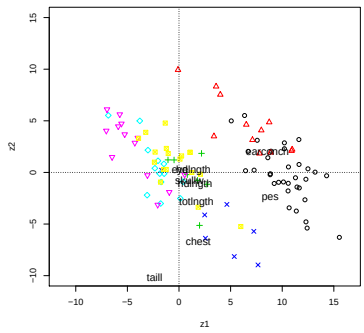
A true story:

- At the start of this study, the researchers thought that they were studying one species of possums. They were very surprised when they saw the results of this analysis.
- It lead them to examine the DNA of the possums, showing them that there were two species of possums. The genomic DNA of the two groups differ in 2.8% (humans and chimpansees differ only in 1%).
- New species: *Trichosurus cunninghamii*, named after the statistician Ross Cunningham

We now redo the analysis, but with doing the matrix manipulations explicitly in R.

```
> possum2<-possum
> possum2[,6:13]<-scale(possum[,6:13],center = TRUE,scale = FALSE)
>
> w<-rep(1/table(possum2$site),table(possum2$site))
> w<-w/sum(w)
>
> m<-manova(cbind(hdlngth,skullw,totlngth,taill,pes,earconch,
+                    eye,chest,belly)~site,
+           data=possum2,weights=w)
> W<-var(m$residuals)
> B<-var(m$fitted.values)
> LDA<-svd(solve(W)%*%B)
```

```
> Z2<-as.matrix(possum2[,6:14])%*%LDA$v
> plot(Z2[,1],Z2[,2],col=possum2$site,pch=possum2$site,asp=1,
+      xlim=c(-10,15),ylim=c(-10,15),
+      xlab="z1",ylab="z2")
> text(x=LDA$v[,1]*13,y=LDA$v[,2]*13,labels=names(possum2)[6:13],cex=1.2)
> abline(h=0,lty=3)
> abline(v=0,lty=3)
```



With high-dimensional predictors, we discuss two solutions:

- replace the $p \times p$ matrices $\boldsymbol{W}$ and $\boldsymbol{B}$ by their diagonal matrices (i.e. put zeroes on the off-diagonal positions)
- Sparse LDA by imposing an L_1-penalty on $\boldsymbol{v}$.

  Two approaches: Zhou *et al.* (2006, Journal of Computational and Graphical Statistics ,**15**, 265-286), and Clemmensen *et al.* (2011, Technometrics, **53**).

4 Cluster Analysis

4.1 Introduction

Cluster analysis is a set of data analysis tools that aims to group data points into clusters such that observations that are *similar* are grouped together in a cluster, and observations that are *dissimilar* do not end up together in the same cluster. There are plenty of clustering methods; here we only discuss a few of the classical methods.

It is important to realise that most of the methods are not build on firm theory. There is no theoretical guarantee that a clustering method gives a good solution. We therefore rather consider cluster analysis as a data exploration tool.

Throughout the course notes we will use the terms *dissimilarity* and *distance* interchangeably. As will become clear, clustering methods start from a distance or dissimilarity matrix. For a sample of size n, the distance matrix is an $n \times n$ matrix which contains the pairwise distances between the observations. However, "distance" is only one way of measuring the dissimilarity between observations. For example, suppose observations are vectors with 0/1 indicators for indicating whether a particular product has been bought in a shop. Then the dissimilarity between two observations can e.g. be expressed as the number of products that are bought by the first person, but not by the second.

Introduction

Objective: grouping of observations into *clusters*, so that

- similar observations appear in the same cluster
- dissimilar observations appear in distinct clusters

$\longrightarrow$ need for a measure for *similarity* and *dissimilarity*

Some examples:

- DNA micro-arrays: $n \times p$ Matrix for which
  - every column contains the expression levels of one of p genes for n subjects
  - every row contains the expression levels of p genes for one subject (*sample*)

  Research question: look for groups of samples that have similar gene expression patterns

  Or, look for groups of genes that have similar expression levels across the different samples. This can help us in understanding the regulation and functionality of the genes.

- Abundance studies: the abundances of n plant species are counted on p plots (habitats)

  Research question: look for groups of species that live in the same habitats, or, look for groups of habitats that have similar species communities

$\longrightarrow$ both *observations* (rows) and *variables* (columns) can be clustered

---

Cluster analysis is a class of algorithms that can be organised into two large groups:

- *Hierarchical methods*: with a further division into *agglomerative* and *divisive* methods

  The solution of a hierarchical clustering is a sequence of $n - 1$ nested cluster configurations.

- *Partition-based methods*:

  These methods start with a given number of clusters (k) and an initial cluster configuration.

  The p dimensional space is partitioned into k subspaces that correspond to k clusters.

There are actually several other types of clustering methods, but in this course we only focus on hierarchical methods and partition-based methods.

4.2 Graphical Diagnostic Tools

Before specific clustering methods are described, we will introduce a few *diagnostic tools* that can help to assess the quality (correctness) of the result of the clustering method. This is important, because clustering methods are basically computational algorithms that classify observations into clusters, but there is no strong theoretical basis that guarantees a good clustering. Therefore it is important to check the result of the clustering algorithm.

Graphical Diagnostic Tools

There is no cluster method that guarantees a good solution

$\longrightarrow$ need for diagnostic tools: *clusplot* and *silhouette plot*.

We describe the two graphical methods using an example (dataset: euro).

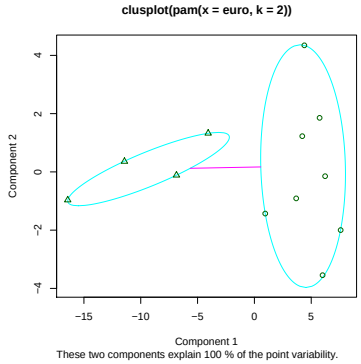
This dataset contains data from 12 European countries that are clustered into 2 groups.

Two variables are measured: gross domestic product (GDP = BBP), and the percentage of the GDP that is attributed to agriculture (data from 1994).

4.2.1 Clusplot

Clusplot

Clusplot: scatter plot in the space of the 2 first PCs



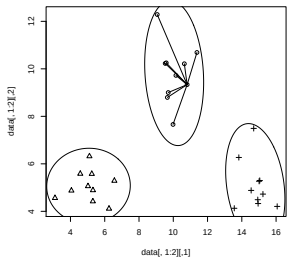
Note that the principal components are not constructed for the purpose of cluster analysis. The scores of the first two dimensions are such that they show maximal variability in a 2-dimensional space (see lecture on PCA).

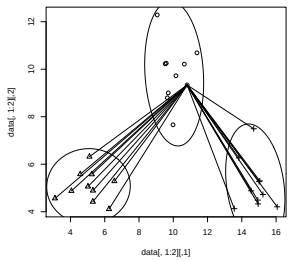
If a clusplot shows nice, well-separated clusters, then this demonstrates that these clusters also exist in the original higher dimensional space. However, if clusters seem to overlap in the clusplot, then there is still a chance that these clusters are separated in the original higher dimensional space (in the clusplot we are looking at the projection of observations into a 2-dimensional space).

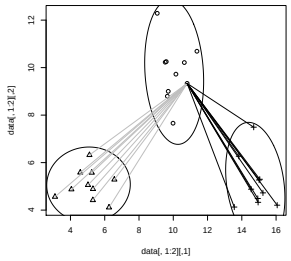
4.2.2 Silhouette Plot

Silhouette Plot Silhouette value of observation i which is in cluster A.

- $\bar{d}(i)$: the average distance between observation i and the other observations in A.
- $\bar{d}(i, C)$: the average distance between observation i and the observations in cluster $C \neq A$.
- $\bar{d}_{\min}(i) = \min_{C \neq A} \bar{d}(i, C)$







The silhouette value of observation i is then defined as

$$s(i) = \frac{\bar{d}_{\min}(i) - \bar{d}(i)}{\max\{\bar{d}(i), \bar{d}_{\min}(i)\}}.$$

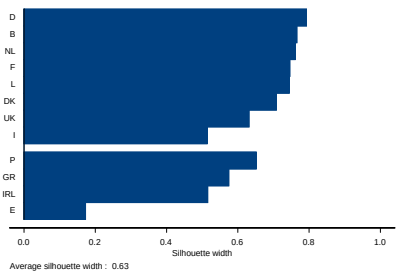
The silhouette value of observation i is then defined as

$$s(i) = \frac{\bar{d}_{\min}(i) - \bar{d}(i)}{\max\{\bar{d}(i), \bar{d}_{\min}(i)\}}.$$

The extremes have a straightforward interpretation: $s(i) \approx +1 \longrightarrow i$ clearly in cluster A. $s(i) \approx 0 \longrightarrow i$ in between clusters A and B. $s(i) \approx -1 \longrightarrow i$ closer to cluster B.

Silhouette plot: silhouette value of each observation is shown as a bar.

The observations are ordered: observations within the same cluster are collected together. Within the same cluster, the observations are ordered from large silhouette values (above) to small silhouette values (down).



A good clustering result will show large (positive) silhouette values for all (or most of the) observations. If some or many of the observations show small or even negative silhouette values, then this is an indication of poor clustering. Indeed, a negative silhouette value suggests that that observation actually is closer to another cluster (i.e. not the cluster it is assigned to by the clustering algorithm).

4.3 Partition-based clustering

The overall idea behind partition-based clustering, is that the p-dimensional space is *partitioned* into a given number of clusters. These methods typically start from a given number of clusters (k) and a set of initial k cluster centres. The algorithm starts with assigning the observations to cluster centres, followed by computing new cluster centres. This procedure is iteratively repeated until convergence.

Partition based clustering

The number of clusters (k) must be specified before the start of the algorithm.

4.3.1 K-means methods

K-means Method

k initial cluster centres $\boldsymbol{c}_1^{(0)}, \ldots, \boldsymbol{c}_k^{(0)}$.

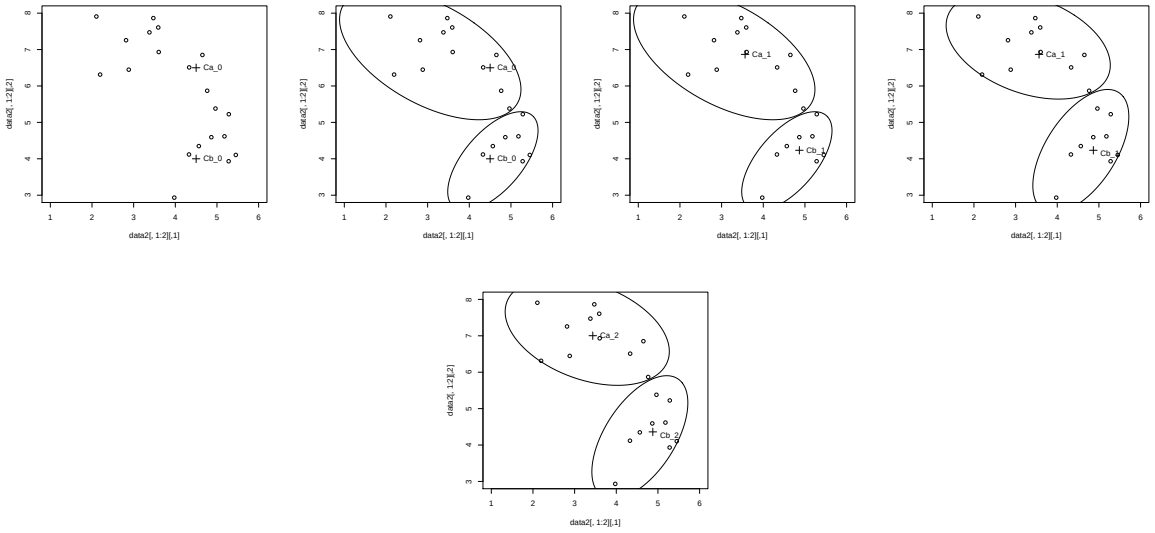
1. assign $\boldsymbol{x}_i$ to cluster j for which

$$d(\boldsymbol{x}_i, \boldsymbol{c}_j^{(s-1)}) \leq d(\boldsymbol{x}_i, \boldsymbol{c}_k^{(s-1)}) \ (k \neq j)$$

   Let $m_i = j$ if observation i is assigned to cluster j. We call m_i the *cluster membership*.

2. recalculate cluster centres $\boldsymbol{c}_j^{(s)}$ as the multivariate sample mean of $\forall x_i$ with $m_i = j$.

3. Cluster centres changed? Yes $\rightarrow$ $s = s + 1$ and go to 1. No $\rightarrow$ stop algorithm



Example Consider the euro dataset

```
> euro.kmeans <- kmeans(euro, centers = 2)
> euro.kmeans
K-means clustering with 2 clusters of sizes 8, 4

Cluster means:
     GNP agricult
1 17.825   4.5625
2  9.000  16.1250

Clustering vector:
  B  DK   D  GR   E   F IRL   I   L  NL   P  UK
  1   1   1   2   2   1   2   1   1   1   2   1
```

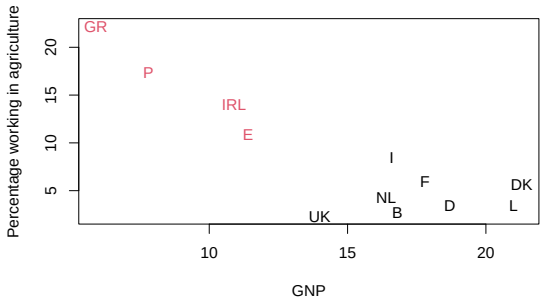
The Euro dataset contains data on the Gross National Product (GNP) per capita and percentage of the population working in agriculture for each country belonging to the European Union in 1993. The dataset is actually named "agriculture" and comes from the R package cluster.

```
Within cluster sum of squares by cluster:
[1] 90.76750 71.91375
 (between_SS / total_SS =  77.6 %)

Available components:

[1] "cluster"      "centers"      "totss"        "withinss"
[5] "tot.withinss" "betweenss"    "size"         "iter"
[9] "ifault"
```

Using the dataset euro we construct a scatter plot of the variables bbp and agriculture with an indication of the cluster membership.



4.3.2 Pam

Pam: Partitioning Around Mediods

- instead of cluster centres (centroids) the method uses *mediods*;

  a mediod of cluster j is one of the observations of cluster j (the most "representative" observation).

- the method is less sensitive to outliers then k-means (means are sensitive to outliers).

Pam

k initial mediods: $\boldsymbol{c}_1^{(0)}, \ldots, \boldsymbol{c}_k^{(0)}$. The s-th step:

1. assign each $\boldsymbol{x}_i$ to cluster j for which

$$d(\boldsymbol{x}_i, \boldsymbol{c}_j^{(s-1)}) \leq d(\boldsymbol{x}_i, \boldsymbol{c}_k^{(s-1)}) \ (k \neq j)$$

2. Or with cluster membership notation

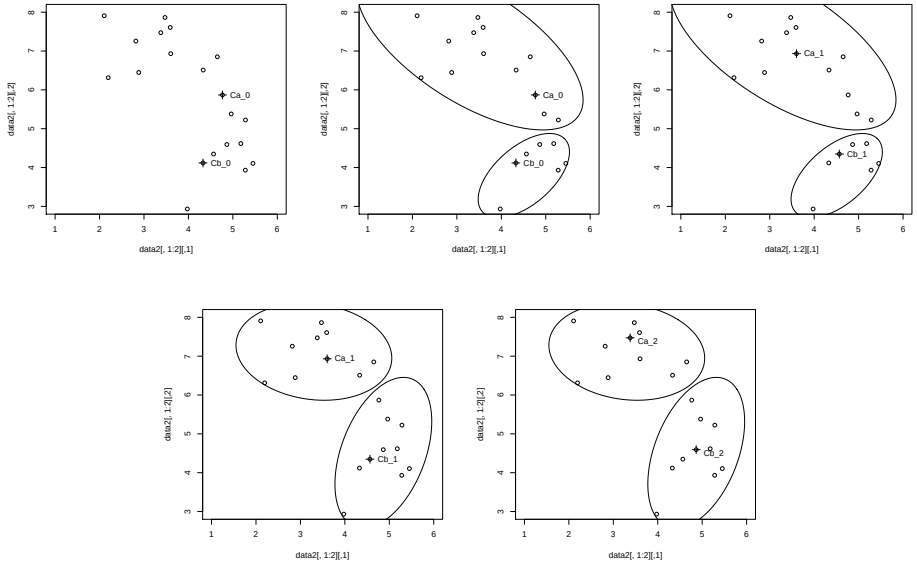
$$d(\boldsymbol{x}_i, \boldsymbol{c}_{m_i}^{(s-1)}) \leq d(\boldsymbol{x}_i, \boldsymbol{c}_j^{(s-1)}) \ (\forall j = 1, \ldots, k)$$

3. search new mediods $\boldsymbol{c}_j^{(s)}$ $(j = 1, \ldots, k)$

$$\text{so that } Q^{(s)} = \sum_{i=1}^{n} d(\boldsymbol{x}_i, \boldsymbol{c}_{m_i}^{(s)})$$

   is minimal. (*objective function*)

4. the algorithms stops if the mediods are converged.



Example Dataset: abundance

The dataset contains the abundances of 50 plant species on 17 locations (plots or habitats).

The data comes from the Steneryd Nature Park in the south of Sweden.

We wish to cluster the 17 habitats. Habitats in a same cluster are more or less homogeneous w.r.t. the composition of the species (species communities).

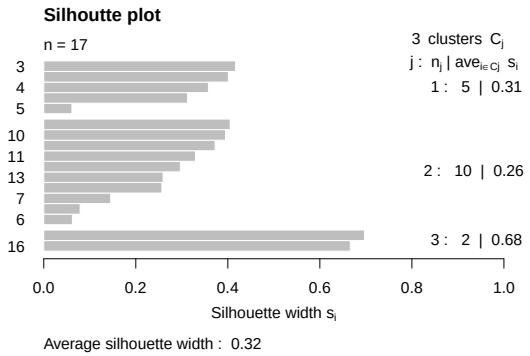
```
> abundance.pam <- pam(abundance[, 2:51], k = 3)
> summary(abundance.pam)
Medoids:
     ID Airpra Teenud Rumtun Viciangus Trifdub
[1,]  4      0      0      0         1       1
[2,] 11      0      0      0         0       0
[3,] 17      0      0      0         0       0
     Visvul Galver Hiepil AchMil Stegram Fesovi
[1,]      5      4      8      9       0     30
[2,]      0      0      0      0       0      4
[3,]      0      0      0      0       0      0
...
          Verhed Ranfic Merper
[1,]      0      0      0
[2,]      0     13      1
[3,]      0     37     45
```

```
Clustering vector:
 [1] 1 1 1 1 1 2 2 2 2 2 2 2 2 2 2 3 3
Objective function:
   build     swap
27.79622 27.79622

Numerical information per cluster:
     size max_diss   av_diss diameter separation
[1,]    5 38.01316 28.645311 57.68882   38.07887
[2,]   10 49.47727 30.989268 60.24948   38.07887
[3,]    2 19.41649  9.708244 19.41649   38.88444
...
Available components:
 [1] "medoids"    "id.med"     "clustering"
 [4] "objective"  "isolation"  "clusinfo"
 [7] "silinfo"    "diss"       "call"
[10] "data"
...
```

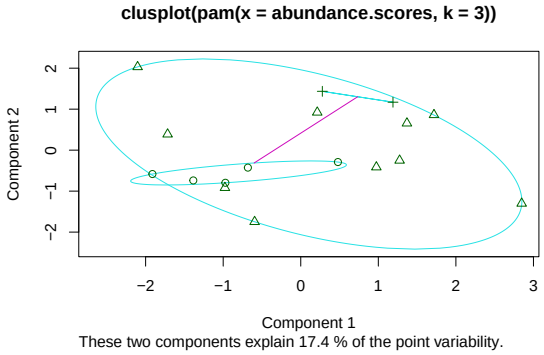
```
plot(silhouette(abundance.pam),
     main="Silhoutte plot")
```

The silhouette plot




```
ab.svd<-svd(abundance[, 2:51])
abundance.scores<-ab.svd$u[,1:17]%*%diag(ab.svd$d[1:17])
abundance.pam2 <- pam(abundance.scores, k = 3)
clusplot(abundance.pam2)
```

The clusplot



In the previous slide the clustering was repeated on the first 17 principal components of the dataset. The reason is that the function clusplot returns an error when directly applied to abundance.pam, because the number of variables (50) is larger than then number of observations (17). We know, however, that this should be no problem. Therefore we appliedd the svd with the svd function and extracted all 17 PCs. Although this seems like a dimension reduction from 50 to 17, we know that the rank of the data matrix is (at most) 17, and hence the 50-dimensional observations live in a 17-dimensional subspace, which is exactly the subspace spanned by the first 17 PCs. Once we have the new data set with the scores on these 17 PCs (abundance.scores), we repeat the clustering with the pam function. The output is not shown, but this will obviously return the same clustering as on the original dataset. Now the clusplot function runs without complaining.

4.4 Hierarchical Cluster Analysis

4.4.1 Agnes

Hierarchical Cluster Analysis: Agnes

General Algorithm of Hierarchical Cluster Analysis

General: in step 0 each observation is considered as a cluster (i.e. n clusters). Every next step consists of:

1. merge the two clusters with the smallest intercluster dissimilarity
2. recalculate the intercluster dissimilarities

In step 0 the intercluster dissimilarity coincides with the dissimilarity between the corresponding observations

$\leftrightarrow$ intercluster dissimilarity?

Let's say that dissimilarity between two observations is simply measured in terms of Euclidean distance. Then, in step 0, clusteres are formed by individual observations and hence distances between cliusters are simply distances between individual observations. Once we start merging clusters (or merging observations to form a cluster), then the question pops up how to measure distances between clusters (or dissimilarities between clusters)? Several options will be discussed.

Intercluster Dissimilarities

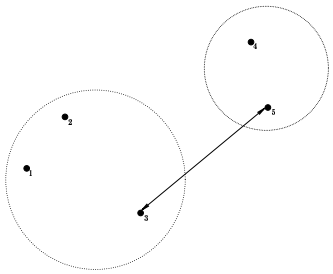
- We represent clusters (e.g. C_1 and C_2) as sets of points $\boldsymbol{x}_i$ which belong to that cluster
- $d(C_1, C_2)$: intercluster dissimilarity between C_1 and C_2

We consider three intercluster dissimilarities.

- Single Linkage = Nearest Neighbour:

$$d(C_1, C_2) = \min_{\boldsymbol{x}_1 \in C_1; \boldsymbol{x}_2 \in C_2} d(\boldsymbol{x}_1, \boldsymbol{x}_2),$$

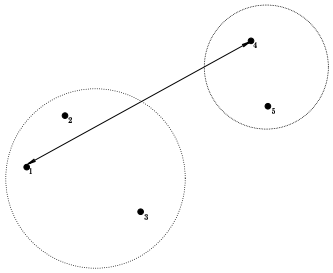
  i.e. the dissimilarity between C_1 and C_2 is determined by the smallest dissimilarity between a point of C_1 and a point of C_2.



- Complete Linkage = Furthest Neighbour:

$$d(C_1, C_2) = \max_{\boldsymbol{x}_1 \in C_1; \boldsymbol{x}_2 \in C_2} d(\boldsymbol{x}_1, \boldsymbol{x}_2),$$

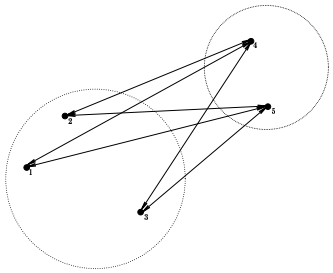
i.e. the dissimilarity between C_1 and C_2 is determined by the largest dissimilarity between a point of C_1 and a point of C_2.



- Average Linkage = Group Average:

$$d(C_1, C_2) = \frac{1}{|C_1||C_2|} \sum_{\boldsymbol{x}_1 \in C_1; \boldsymbol{x}_2 \in C_2} d(\boldsymbol{x}_1, \boldsymbol{x}_2),$$

i.e. the dissimilarity between C_1 and C_2 is determined by the average dissimilarity between all points of C_1 and all points of C_2.



4.4.2 Cluster Tree

Cluster Tree

hierarchical nature of the algorithm: nested sequence of clusters

$\longrightarrow$ visualisation via a tree

The height of the branches indicates the intercluster dissimilarity at which the clusters are merged. This can be used as an instrument for deciding the number of clusters in the data.

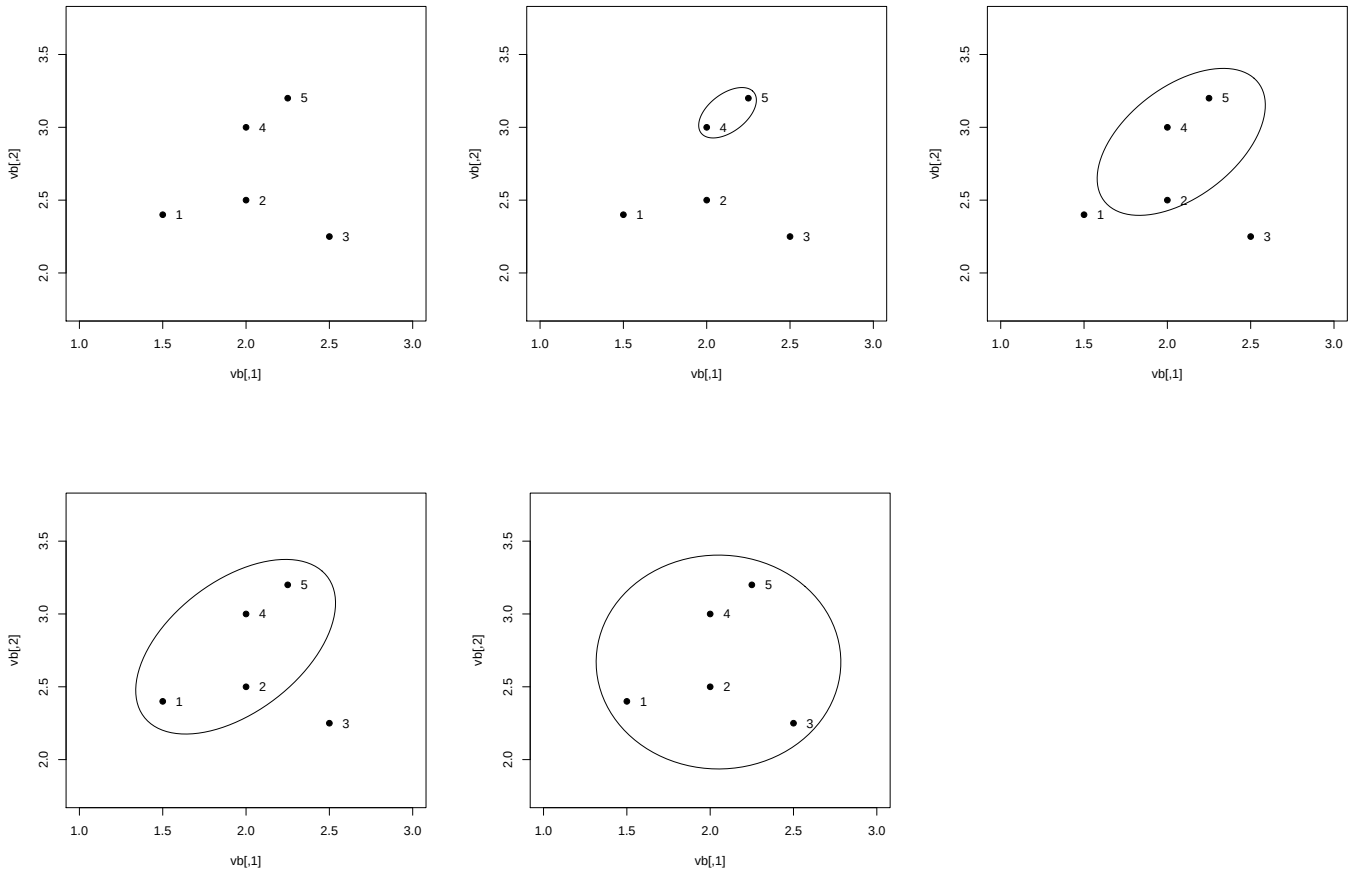
We continue with an example. The next slide gives the 2-dimensional data (X_1 and X_2) and the distance matrix $\boldsymbol{D}$.

X_1	X_2
1.50	2.40
2.00	2.50
2.50	2.25
2.00	3.00
2.25	3.20

$$\boldsymbol{D} = \begin{bmatrix} 0.00 & 0.51 & 1.01 & 0.78 & 1.10 \\ 0.51 & 0.00 & 0.56 & 0.50 & 0.74 \\ 1.01 & 0.56 & 0.00 & 0.90 & 0.98 \\ 0.78 & 0.50 & 0.90 & 0.00 & 0.32 \\ 1.10 & 0.74 & 0.98 & 0.32 & 0.00 \end{bmatrix}.$$

The next slide shows the data, and the merging of clusters according to single linkage.

Single Linkage=Nearest Neighbor



$$\boldsymbol{D} = \begin{bmatrix} 0.00 & 0.51 & 1.01 & 0.78 & 1.10 \\ 0.51 & 0.00 & 0.56 & 0.50 & 0.74 \\ 1.01 & 0.56 & 0.00 & 0.90 & 0.98 \\ 0.78 & 0.50 & 0.90 & 0.00 & 0.32 \\ 1.10 & 0.74 & 0.98 & 0.32 & 0.00 \end{bmatrix}$$

Single Linkage=Nearest Neighbor

```
> vb.agnes <- agnes(vb, method = "single")
> summary(vb.agnes)
Object of class 'agnes' from call:
 agnes(x = vb, method = "single")
Agglomerative coefficient:  0.2096014
Order of objects:
[1] 1 2 4 5 3
Merge:
     [,1] [,2]
[1,]   -4   -5
[2,]   -2    1
[3,]   -1    2
[4,]    3   -3
Height:
[1] 0.5099020 0.5000000 0.3201562 0.5590170
...
> plot(vb.agnes, which = 2)
```

The R output shows the 4 steps in this hierarchical clustering (the merge part of the output).

In step 1 we read: -4 -5
This tells us that observations 4 and 5 are merged into a cluster.

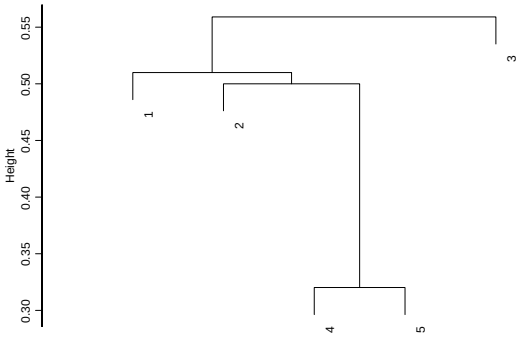
In step 2 we read: -2 1
This tells us that observation 2 is merged with the cluster formed in step 1 (observations are referred to by the minus sign, and previously formed clusters are referred to with positive numbers).

In step 3 we read: -1 2
This tells us that observation 1 (mind the minus sign) is merged with the cluster formed in step 2.

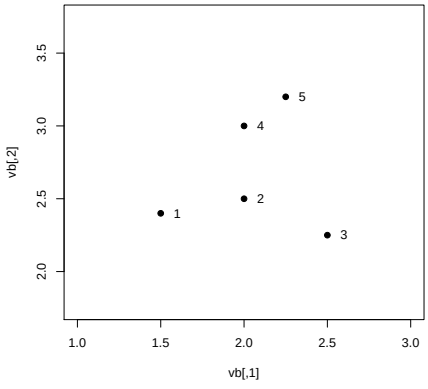
In step 4 we read: 3 -3
This tells us that the cluster formed in step 3 is merged with observation 3 (mind the minus sign).

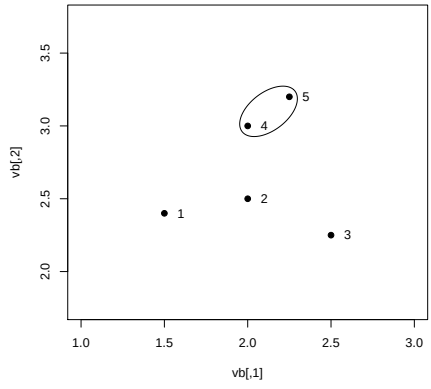
Single Linkage=Nearest Neighbour

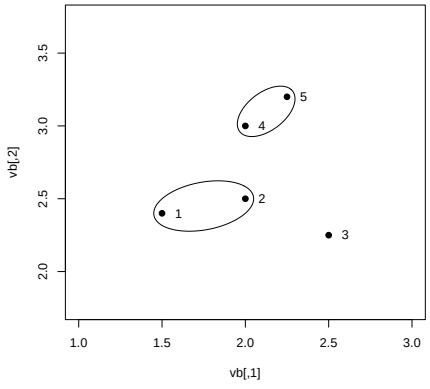
$$\boldsymbol{D} = \begin{bmatrix} 0.00 & 0.51 & 1.01 & 0.78 & 1.10 \\ 0.51 & 0.00 & 0.56 & 0.50 & 0.74 \\ 1.01 & 0.56 & 0.00 & 0.90 & 0.98 \\ 0.78 & 0.50 & 0.90 & 0.00 & 0.32 \\ 1.10 & 0.74 & 0.98 & 0.32 & 0.00 \end{bmatrix}$$

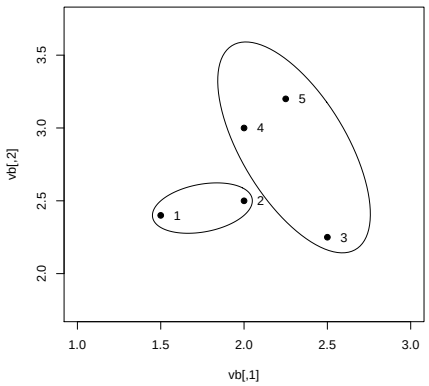


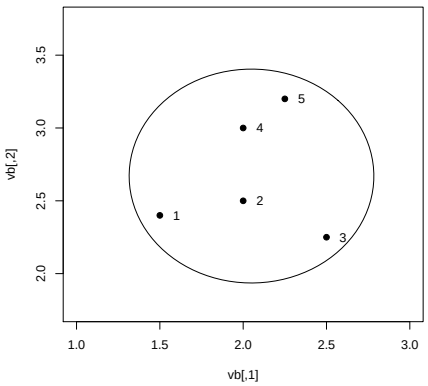
Complete Linkage=Furthest Neighbour











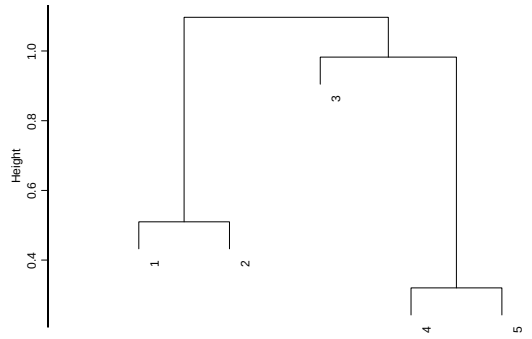
$$\boldsymbol{D} = \begin{bmatrix} 0.00 & 0.51 & 1.01 & 0.78 & 1.10 \\ 0.51 & 0.00 & 0.56 & 0.50 & 0.74 \\ 1.01 & 0.56 & 0.00 & 0.90 & 0.98 \\ 0.78 & 0.50 & 0.90 & 0.00 & 0.32 \\ 1.10 & 0.74 & 0.98 & 0.32 & 0.00 \end{bmatrix}$$

Complete Linkage = Furthest Neighbour

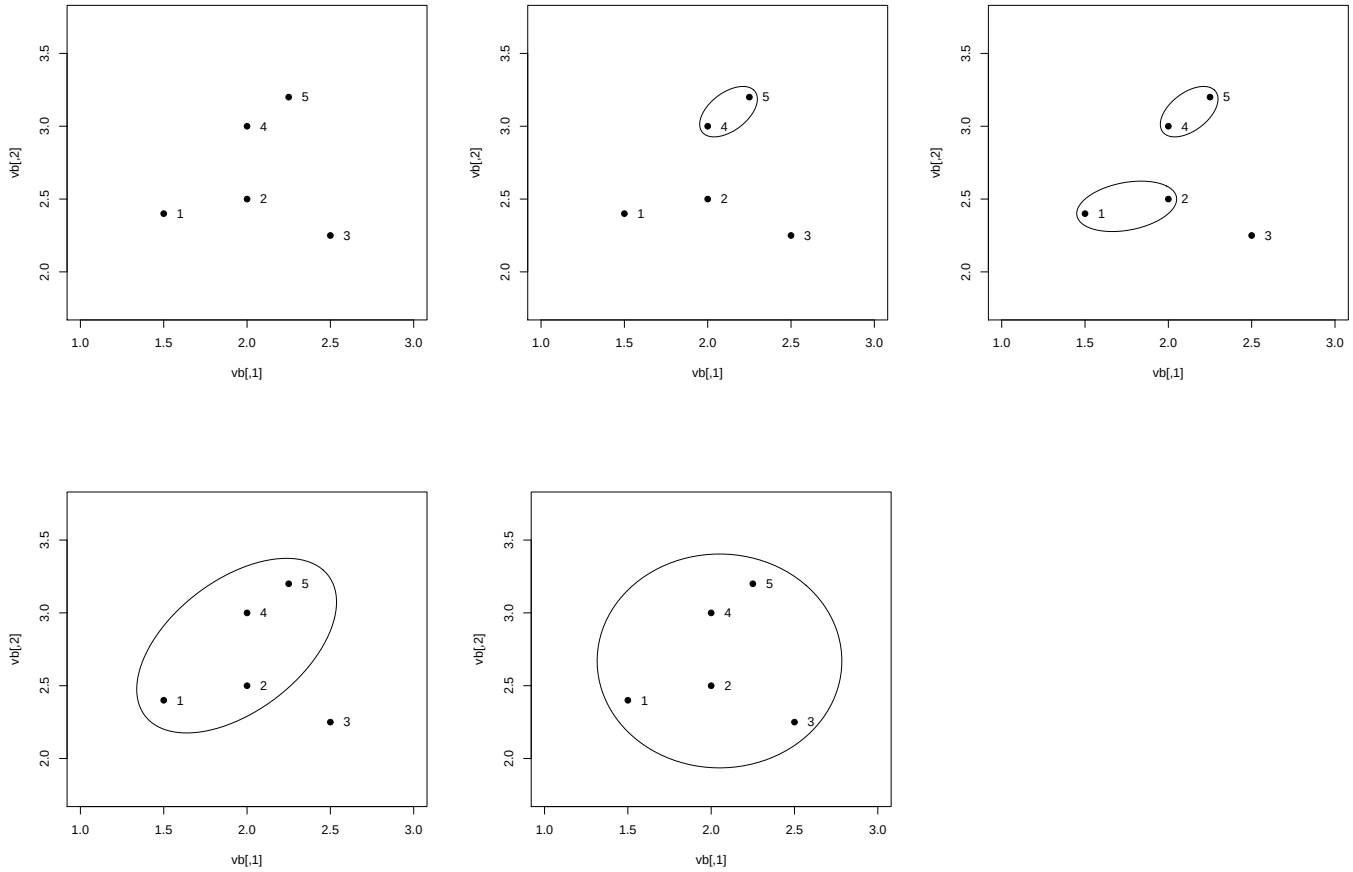
```
> vb.agnes <- agnes(vb, method = "complete")
> summary(vb.agnes)
Object of class 'agnes' from call:
 agnes(x = vb, method = "complete")
Agglomerative coefficient:  0.5180567
Order of objects:
[1] 1 2 3 4 5
Merge:
     [,1] [,2]
[1,]   -4   -5
[2,]   -1   -2
[3,]   -3    1
[4,]    2    3
Height:
[1] 0.5099020 1.0965856 0.9823441 0.3201562
...
> plot(vb.agnes, which = 2)
```

Complete Linkage=Furthest Neighbour

$$\boldsymbol{D} = \begin{bmatrix} 0.00 & 0.51 & 1.01 & 0.78 & 1.10 \\ 0.51 & 0.00 & 0.56 & 0.50 & 0.74 \\ 1.01 & 0.56 & 0.00 & 0.90 & 0.98 \\ 0.78 & 0.50 & 0.90 & 0.00 & 0.32 \\ 1.10 & 0.74 & 0.98 & 0.32 & 0.00 \end{bmatrix}$$



Average Linkage=Group Average



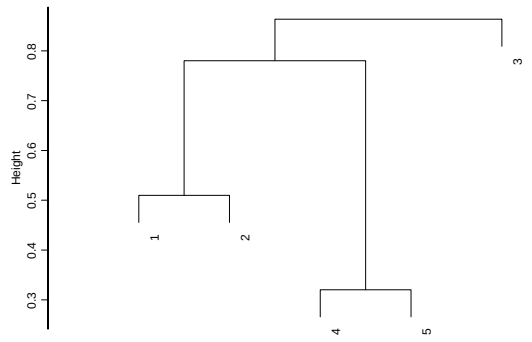
$$\boldsymbol{D} = \begin{bmatrix} 0.00 & 0.51 & 1.01 & 0.78 & 1.10 \\ 0.51 & 0.00 & 0.56 & 0.50 & 0.74 \\ 1.01 & 0.56 & 0.00 & 0.90 & 0.98 \\ 0.78 & 0.50 & 0.90 & 0.00 & 0.32 \\ 1.10 & 0.74 & 0.98 & 0.32 & 0.00 \end{bmatrix}$$

Average Linkage = Group Average

```
> vb.agnes <- agnes(vb, method = "average")
> summary(vb.agnes)
Object of class 'agnes' from call:
 agnes(x = vb, method = "average")
Agglomerative coefficient:  0.4154842
Order of objects:
[1] 1 2 4 5 3
Merge:
     [,1] [,2]
[1,]   -4   -5
[2,]   -1   -2
[3,]    2    1
[4,]    3   -3
Height:
[1] 0.5099020 0.7802285 0.3201562 0.8634841
...
> plot(vb.agnes, which = 2)
```

Average Linkage= Group average

$$\boldsymbol{D} = \begin{bmatrix} 0.00 & 0.51 & 1.01 & 0.78 & 1.10 \\ 0.51 & 0.00 & 0.56 & 0.50 & 0.74 \\ 1.01 & 0.56 & 0.00 & 0.90 & 0.98 \\ 0.78 & 0.50 & 0.90 & 0.00 & 0.32 \\ 1.10 & 0.74 & 0.98 & 0.32 & 0.00 \end{bmatrix}$$



4.4.3 Ward's Method

Ward's Method

Ward's cluster method is an agglomerative hierarchical cluster method which is not based on intercluster dissimilarities. This method is also known under the name "*trace* method".

The merging of 2 clusters is based on the sum of squares within a cluster C. Let $\boldsymbol{x} = (x_1 \dots x_p) \in C$, then the sum of squares is

$$S(C) = \sum_{\boldsymbol{x} \in C} \sum_{i=1}^{p} (x_i - \bar{x}_{Ci})^2$$

where $\bar{x}_{Ci}$ is the average of the variable i within cluster C.

Two clusters C_i and C_j are merged if their increase in the sum of squares, i.e.

$$S(C_i \cup C_j) - (S(C_i) + S(C_j)),$$

is minimal.

Example:

```
vb.agnes<-agnes(vb,method="ward")
```

4.5 Determination of the Number of Clusters

4.5.1 Introduction

Determination of the Number of Clusters

Introduction

A general procedure: try first a few hierarchical cluster analyses and select a series of sensible k. For these few values of k, partition based methods are applied to find cluster configurations. By means of e.g. diagnostic plots, the final cluster configuration can be selected.

4.5.2 Scree Plot

Scree Plot with Wilks' Lambda

Wilks' Lambda: test statistic for detecting unequal multivariate means (MANOVA / LDA).

$\longrightarrow$ we expect that the cluster centres (means) are distinct

This means: the variability of the observations within a cluster is smaller than the variability between the cluster centres. This is exactly what is measured by Wilks' Lambda,

$$\Lambda = \left(\frac{|\boldsymbol{W}|}{|\boldsymbol{B} + \boldsymbol{W}|} \right)^{2/n},$$

where $\boldsymbol{W}$ and $\boldsymbol{B}$ are the matrices of the sum of squares and cross products within and between the clusters, respectively (see lecture on LDA). (the exponent $2/n$ is not necessary for our current purpose).

The matrices $\boldsymbol{B}$ and $\boldsymbol{W}$ were introduced in the lecture on LDA (Fisher's discriminant analysis). The can be interpreted as the multivariate versions of the between and within group variability as in ANOVA. A small value of Λ indicates that between group (cluster) variability is large as compared to within group variability, indicating a good clustering.

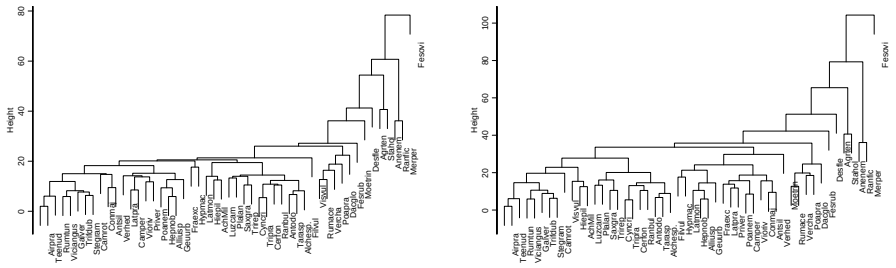
Λ can be computed in R by either computing $\boldsymbol{W}$ and $\boldsymbol{B}$ by means of the lda function, or Wilks' Λ can be computed directly from the manova function ("manova" stands for multivariate ANOVA).

If we have a series of cluster solutions, e.g. $k = 1, \ldots, K$, we compute for each solution the Wilks' Lambda (Λ_k), and we construct a scree plot of the Λ_k versus k. We illustrate this by means of an example.

Example Consider the abundance dataset, but now the transposed matrix so that we can cluster the plant species (dataset: abundanceT). Plant species in the same cluster are more or less homogeneous w.r.t. habitats.

- In a first phase we perform a hierarchical clustering.

```
> abundance.agnes.ave <- agnes(abundanceT[, 2:18], method =
  "average")
> plot(abundance.agnes.ave)
> abundance.agnes.com <- agnes(abundanceT[, 2:18], method =
 "complete")
> plot(abundance.agnes.com)
```



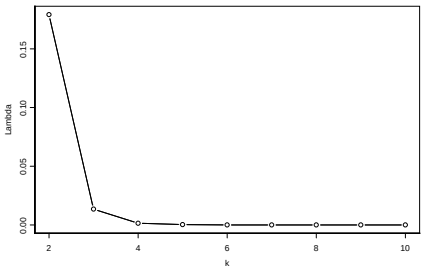
$\longrightarrow$ compute Wilks' Lambda for $k = 2, \ldots, 10$ clusters

- We perform thus 9 partition-based cluster analyses, and 9 MANOVAs to find the Wilks' Lambda's.

```
abundance.scree<-as.data.frame(matrix(ncol=2,nrow=9))
names(abundance.scree)<-c("k","Lambda")
abundance.scree$k<-2:10
```

Next we perform the 9 cluster analyses (PAM) and the 9 MANOVAs.

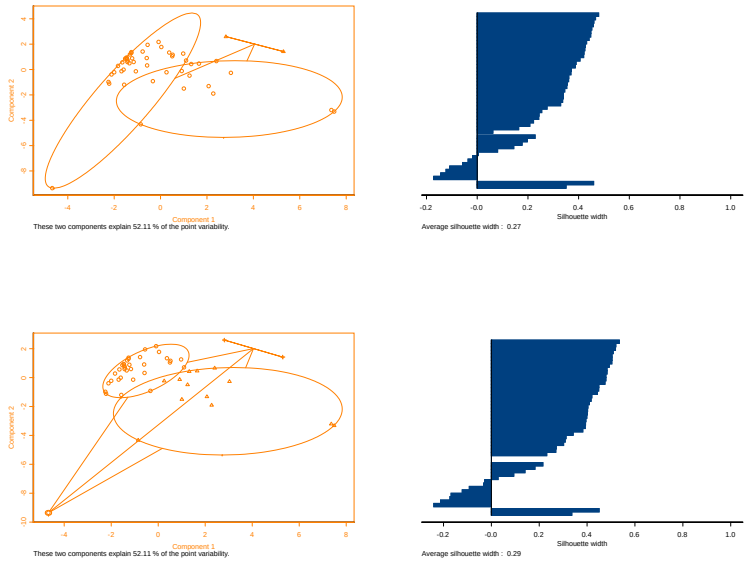
```
> for(k in 2:10) {
        abundance.pam <- pam(abundanceT[, 2:18], k = k)
        abundanceT$clust <- as.factor(abundance.pam$clust)
        tmp <- manova(cbind(Site1, Site2, Site3, Site4, Site5,
                Site6, Site7, Site8, Site9, Site10, Site11, Site12,
                Sit13, Site14, Site15, Site16, Site17) ~ clust,
                data = abundanceT)
        abundance.scree$Lambda[k - 1] <- summary(tmp,
                test = "Wilks")$stats[3]
}
```



The scree plot clearly shows a sharp decrease van Λ_k. After $k = 3$ the plot shows almost no decrease in Λ_k anymore. We conclude to choose $k = 3$ or $k = 4$.

- Finally we construct clusplots and silhouette plots for the PAM solutions for $k = 3$ and $k = 4$ clusters..

```
> abundance.pam <- pam(abundanceT[, 2:18], k = 3)
> plot(abundance.pam)
> abundance.pam <- pam(abundanceT[, 2:18], k = 4)
> plot(abundance.pam)
```



None of the two solutions seems to be very good. For $k = 3$ there is a cluster of only two observations and this small cluster seems to be close to the second cluster (some of the silhouette values of the second cluster are negative). Anyway, this may be an indication to have a closer look at these two observations (outliers?).

The solution for $k = 4$ is similar to the $k = 3$ solution, but it even contains a cluster with a single observation. A "cluster" with a single observation can be hardly called a cluster; it is rather to be considered as an outlying observation that cannot be clustered anywhere else. For such a single-observation cluster, a silhouette value cannot be computed.